

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Selective disclosure approaches in Self-Sovereign Identity: an experimental comparison

ANDREA DE SALVE¹, ANDREA LISI², MAURIZIO CASCINO³, PAOLO MORI², and LAURA RICCI⁴

¹National Research Council (CNR) - ISASI, Via Monteroni, 73100 Lecce, Italy (e-mail: andrea.desalve@isasi.cnr.it)

²National Research Council (CNR) - IIT, Via G. Moruzzi 1, 56124 Pisa, Italy (e-mail: paolo.mori@iit.cnr.it)

³Advancia Integration, Via Monte Procinto 2, 55049 Viareggio (LU), Italy (e-mail: mc.cascino@gmail.com)

⁴Department of Computer Science, University of Pisa, Largo Bruno Pontecorvo 3, 56127 Pisa, Italy (e-mail: laura.ricci@unipi.it)

Corresponding author: Andrea De Salve (e-mail: andrea.desalve@isasi.cnr.it).

ABSTRACT The Self-Sovereign Identity (SSI) paradigm empowers individuals to manage their own credentials by storing them in digital wallets under their full control. In SSI, a credential embeds a list of claims representing personal attributes of the holder, and it is digitally signed by the issuer. When requested, the holder submits some of their credentials to a verifier to prove some of their attributes (e.g., being of legal age). However, to protect their privacy, the holder might want to disclose to the verifier only a subset of the claims embedded in each of the credentials they share, thus requiring a mechanism for hiding the other claims without invalidating the digital signatures of the credentials.

This paper surveys the approaches enabling a subject to disclose only selected claims from a signed credential, known as Selective Disclosure. These approaches define the protocol the issuer should follow to construct the credentials, and the holder should follow to disclose only the claims they wish. We examined five approaches, based on: atomic credentials, hashing, encryption, hash trees, and signature, and we compare them from both a qualitative and quantitative point of view. In order to perform an unbiased quantitative evaluation, we developed a reference implementation for each approach and we assessed their performance in executing the same SSI operations on the same credentials. In particular, we measured the time required to create and verify credentials and presentations, as well as the size they take, varying both the number of claims in the credentials and the percentage of disclosed ones in the presentations. The proposed qualitative and experimental comparison is meant to support application designers in understanding which of the available selective disclosure approach is more convenient for their application scenarios.

INDEX TERMS Self-Sovereign Identity, Selective Information Disclosure, Digital Identity, Data Privacy, Blockchain, Identity Management

I. INTRODUCTION

TODAY'S society is heavily based on the issuance of credentials to individuals to identify them and certify their features or capabilities. For instance, credentials could represent job qualifications, such as Project Management Professional Training Course certificates, or citizenship information, such as ID cards. Often, credentials are used to determine the permissions of individuals to access a location or a service, either physically or digitally. Physical credentials are typically signed and issued by recognized entities (e.g., the local municipality) to individuals (e.g., the citizens), who, in turn, show them when asked by a third party (e.g., a police officer) regulating the access to the location or service. In digital word, the identity management models currently adopted are typically centralized, and digital credentials are issued by Identity Providers (IdPs), e.g., a social network or a

cloud service provider, control the information associated to users' identities. When requested by third parties, for example by a web site where an user is registering (e.g., using the OAuth protocol [1]), such IdPs release the digital credentials related to the user.

As a consequence, the digital information about the identity of an individual is scattered among such identity providers, who have the control on them. Therefore, the risk of user information leakage raises, which leads to privacy issues for the individuals since centralized identity providers represent targets for attacks or could voluntarily disclose such information to other parties.

The Self-Sovereign Identity (SSI) [2] paradigm aims at giving to individuals the control of their digital credentials as they were physical, so that they can choose the ones to show to third parties without the involvement of any other actor. This

is achieved by representing digital credentials as Verifiable Credentials¹, which are digitally signed by the issuers, and delivered to the individuals they refer to, i.e., the subjects of such credentials. Subjects create Verifiable Presentations by selecting a subset of their Verifiable Credentials and digitally sign them. Verifiable Presentations are then sent to third parties (the verifiers) who verify their authenticity before accepting and processing them. Each subject stores the Verifiable Credentials they receive using a wallet application [3] under their control, and when they create a Verifiable Presentation to be sent to a verifier, they select only the credentials they want to show to such verifier. This allows the subject to control the amount of information they disclose to verifiers. Covid-19 vaccination certificates in Europe around 2021 [4] were a real-world use case that resembled this paradigm. For example, in Italy the certificate was digitally signed by the Ministry of Health, issued and stored in the citizen's smartphone, and shown at the entrance of hospitals to IoT devices, which verify the validity of the certificate without any direct interaction with the issuer.

One of the most important properties that Verifiable Credentials should guarantee is the principle of data minimization, i.e., they should allow the subject to make fine-grained decisions about the subset of information of a credential to share with a verifier [5], in order to protect subjects' privacy by not sharing more information than the strictly requested ones. Let us take as an example an individual in the physical world that, buying certain products in a shop, must prove they have the legal age. Showing the ID card reveals the age, i.e., the requested data, but also name, surname, and other personal information to the cashier, although individuals may not be willing to share them. Similarly, in the digital world, sending a digital credential to a verifier, the subject is forced to reveal all the information embedded in such credential, although only some of them are actually requested by such verifier. In general, there are several important features that Verifiable Credentials should achieve for what concerns security and privacy, such as private holder binding [6], signature blinding [7], privacy preserving predicates [8], and selective disclosure [5]. In this paper, we focus on *selective disclosure*, i.e., the capability of Verifiable Credentials to support a fine-grained selection of the information to be disclosed.

A. MOTIVATION

The increasing widespread of SSI-oriented solutions and the need for users to control the disclosure of the data embedded in their Verifiable Credentials led researchers and experts to tackle the problem of selective disclosure in different ways. Selective disclosure solutions perform a transformation of the clear data embedded in a Verifiable Credential in such a way that the subject of the credential can decide which subset of such data a verifier can understand. As a consequence, SSI systems may suffer from performance issues that arise due to

the overhead introduced by selective disclosure mechanisms [9], affecting all the actors of the scenario. Deciding which approach is the most suitable to support selective disclosure in each specific application scenario is a critical task which requires a clear understanding of the strengths and limitations of each approach. Some of the scenario dependent factors that may influence the choice of the selective disclosure approach are the context in which the credentials should be used, the requirements of the subject, and of the verifier. For example, the more attributes a Verifiable Credential has, the larger and more complex it becomes, and this could affect how fast and how much storage it requires to be verified. For this reason, it is very important to determine which selective disclosure approach is more appropriate for the specific use case and context. Different approaches may have different trade-offs in terms of efficiency, security, and usability. Another factor that could influence the choice of the selective disclosure approach to be adopted is the device where the SSI framework will be executed. For example, smartphones holding individual's digital identity information or other IoT devices collecting data from sensors are expected to use lightweight and efficient approaches to guarantee selective disclosure of data because they need to keep energy and memory consumption low. In contrast, recent personal computers and laptops are generally suitable to support computationally heavy approaches for selective disclosure of Verifiable Credentials, provided that they introduce features or capabilities which are important for the end-user experience.

As discussed in Section II, the scientific literature on SSI has grown rapidly in recent years. Several articles have already surveyed the definitions and concepts of SSI, the use of SSI in specific contexts, the challenges and opportunity of SSI, the architectural patterns, the privacy and security properties of SSI, and the frameworks for comparing SSI systems. Many survey papers also take into account selective disclosure as a fundamental technique for preserving the privacy of user's identity information in SSI.

However, to the best of our knowledge, a quantitative assessment of the impact of the different selective disclosure approaches currently available in the scientific literature on the performance of the SSI operations has never been addressed. This gap in the literature highlights the need for comprehensive studies that evaluate how the available selective disclosure mechanisms affect the SSI operations execution time, the required storage space for Verifiable Credentials and Presentations and, consequently, the user experience, taking into account all the actors of the SSI scenario. While the theoretical complexity of selective disclosure mechanisms may suggest varying levels of computational and storage overhead, it may not be sufficient to accurately estimating the practical impact of a selective disclosure approach. Indeed, the selection of an appropriate selective disclosure approach is a very complex task, as it involves multiple factors, including the number of claims embedded in a credential, the proportion of claims to be disclosed, and the specific cryptographic primitives employed.

¹<https://www.w3.org/TR/vc-data-model-2.0/> [Accessed on 10 November 2025]

B. CONTRIBUTION

This paper contributes to the study of the selective disclosure problem in SSI by classifying, implementing, and evaluating a set of approaches that can be used in practice to ensure fine-grained exposure of the information embedded in Verifiable Credentials. Given the numerous approaches proposed to solve the challenge of selective disclosure in Verifiable Credentials, a comprehensive analysis that considers both their theoretical and practical implications is therefore necessary. Moreover, a gap remains in the availability of reference implementations, which are essential for validating each approach and demonstrating their feasibility and effectiveness in real-world applications.

For this reason, this paper provides a suite of reference implementations for each selective disclosure approach, exploiting JSON Web Tokens as data model to encode Verifiable Credentials and the Ethereum blockchain as verifiable data registry for the DID resolution. Since each of these approaches is based on a different technique, this paper adapts each of them to the Verifiable Credentials data model by defining the algorithms, cryptographic hash functions, and primitives exploited by the approaches to enable selective disclosure. All implementations are developed within a unified framework that simplifies the evaluation of each approach using a consistent benchmarking methodology.

Finally, the overhead introduced to enable selective disclosure is evaluated by measuring the time required by each approach to create and verify credentials and presentations and the related dimensions. In addition, the strengths and weaknesses of each technique are investigated by varying several parameters related to both the specific approach and the credential.

The main contributions of our work are the following:

- We provide a comprehensive identification, classification, and conceptualization of existing approaches for selective disclosure that facilitate the understanding of their design principles and the comparisons across different techniques.
- We provide a reference implementation of each approach by using JSON Web Tokens (JWTs) to represent Verifiable Credentials and the Ethereum blockchain as a verifiable data registry, thereby demonstrating their practical applicability.
- We experimentally assess the impact of each approach, measuring the size of Verifiable Credentials and Presentations and the time required to generate and verify them across various parameters, such as the key length, hash function, encryption algorithm, number of claims, and percentage of disclosed claims, offering valuable insights for academia and industry practitioners.
- We provide a detailed discussion of the results of each approach, setting the stage for future enhancements in the field.

C. PAPER STRUCTURE

The paper is structured as follows: Section II presents some related work; Section III describes the background information to understand the paper; Section IV provides a classification of selective disclosure strategies; Section V aggregates the information common to all the approaches used in this comparison; Section VI describes the selective disclosure approaches; Sections VII and VIII present the experimental setups and their evaluation, respectively; Section IX discusses the usability and the performance of each approach, while Section X concludes the paper and suggests some future research directions.

II. RELATED WORK

The scientific literature on SSI has grown rapidly in recent years, investigating the SSI from different points of view. In this section, we exploit the categorization scheme proposed in [17] in order to classify the related works about SSI and describe how the proposed research addresses previously unconsidered weaknesses or gaps related to the problem of selective disclosure in SSI. Table 1 summarizes the key differences between the proposed research and existing surveys in current literature. Each relevant survey published in a journal or conference proceeding has been categorized according to its main contribution, the domain of the survey, the research type and the research method, whether the paper covers the topic of selective disclosure (Sel. Disclosure), and whether it includes an experimental evaluation (Experimental Eval.).

Authors of [2], [10]–[13] investigate the definitions & concepts to the SSI by focusing on the terminology and the components of the SSI architecture [2], on designing a taxonomy of the existing definitions of the SSI [10], on the properties, standards, frameworks, regulations for the SSI [11], on the requirements addressed by SSI in enterprise [13], and on the comparison of the principles of the SSI with the properties of other identity management models [12]. In this category of contribution, the authors of [12] recognize the selective disclosure as a comparison point of the identity management models, while authors of [13] assert that the selective disclosure is a fundamental requirement for enterprises.

The majority of survey papers [14]–[21] focused on investigating the use of SSI in a specific context and they do not consider the topic of selective disclosure. Indeed, most of them investigate the use of the SSI in the Industrial Internet of Things domain [14], in healthcare [15], [18], in cloud and WSN [20].

The authors of [17], [21] focused on the use of SSI in academic context by providing a systematic mapping study [17] and a bibliometric analysis [21] of the scientific production. Authors of [16] provide a survey on the use of SSI in relevant research initiatives, while in [19] a systematic review and mapping of the scientific and non-scientific literature on SSI is provided.

The challenges and opportunity provided by the SSI has been investigated by [22] in the context of Internet of Things (IoT) and by [23], which provides a mapping of the challenges

Contributions	Paper	Domain	Research Type - Research Method	Sel. Disclosure	Experimental Eval.
Definitions & Concepts	[2]	general	Conceptual Paper- Property analysis & synthesis	✗	✗
	[10]	general	Conceptual Paper - Taxonomy	✗	✗
	[11]	general	Validation research - Systematic Literature Review	✗	✗
	[12]	general	Validation & Comparison - Property analysis & Synthesis	✓	✗
	[13]	enterprises	Validation & Comparison - Systematic Literature Review	✓	✗
Use of SII	[14]	IoT	Validation & Comparison - Property analysis & Synthesis	✗	✗
	[15]	healthcare	Validation research - Property analysis & Synthesis	✗	✗
	[16]	general	Conceptual Paper - Literature Review	✗	✗
	[17]	general	Evaluation research - Systematic Literature Review	✗	✗
	[18]	healthcare	Validation research - Systematic Literature Review	✗	✗
	[19]	general	Validation research - Systematic Literature Review	✗	✗
	[20]	cloud, IoT, WSN, health	Validation & Comparison - Property analysis & Synthesis	✗	✗
	[21]	general	Validation research - Systematic Literature Review	✗	✗
Challenges & Opportunity	[22]	IoT	Evaluation research - Property analysis & Synthesis	✗	✗
	[23]	general	Validation research - Taxonomy	✓	✗
Architectural Patterns	[24]	general	Validation research - Property analysis & Synthesis	✓	✗
	[25]	general	Validation research - Property analysis & Synthesis	✓	✗
Framework	[26]	general	Validation & Comparison - Property analysis & Synthesis	✗	✗
	[27]	general	Evaluation research - Property analysis & Synthesis	✓	✗
	[28]	general	Validation research - Property analysis & Synthesis	✓	✗
	[29]	general	Validation & Comparison - Property analysis & Synthesis	✗	✗
	[30]	general	Validation & Comparison - Property analysis & Synthesis	✗	✗
Privacy & Security	[31]	general	Validation research - Threat modeling and analysis	✓	✗
	[32]	general	Validation research - Systematic Literature Review	✓	✗
	our	general	Evaluation research - Experiment/Simulation	✓	✓

TABLE 1. Comparison of key differences between this paper and the existing SSI related surveys in the current literature

into different layer of the SSI architecture and recognizes the selective disclosure as a challenge of the credential layer.

The architectural patterns used in SSI have been investigated in [24], [25]. They identify a pattern for selective content generation and mention the corresponding solutions without evaluating them.

Another large part of the survey papers focused on the definition of frameworks for the validation, evaluation, and comparison of the properties of SSI systems and identity models [26]–[30]. In particular, the authors of [26] compare existing implementations and properties of blockchain-based SSI, while the authors of [28] collect and evaluate SSI properties defined in the literature (including the selective disclosure). A framework for the comparison of the properties of SSI with the properties of other identity models has been proposed by [29], [30], while authors of [27] identify and compare the non-functional and functional requirements of SSI systems by specifying whether the ability to derive/share credential is supported, not supported, or partially supported.

The privacy & security of SSI systems have been relatively under-investigated, with only a few contributions to date [31], [32]. Finally, authors of [32] conduct a systematic literature review in order to identify the general methods used to guar-

antee selective disclosure of credentials. In particular, they identify selective disclosure approaches and classify each approach according to the credential type, the use of blockchain, and the use of Zero-Knowledge.

Unfortunately, the topic of selective disclosure has been marginally discussed in several survey papers, and no research papers specifically focusing on evaluating the selective disclosure approaches. The classification and review of selective disclosure solutions, such as [32], is useful to understand current research lines and the general characteristics of each solution, but to achieve full utilization and deep understanding, the performance of the different selective disclosure approaches must be evaluated. Indeed, in order to design identity management solutions that meet specific performance and privacy criteria, it is important to evaluate which approach is the most suitable for selective disclosure. Measurement of the performance of the selective disclosure solutions and the assessment of the overhead introduced by each approach is a critical task, which has not been investigated before. This paper contributes to fill this gap by categorizing, implementing and experimentally evaluating the approaches that can be used in practice to ensure fine-grained exposure of the information embedded in credentials.

III. BACKGROUND

A. SELF-SOVEREIGN IDENTITY COMPONENTS

The SSI framework is characterized by a number of basic components [2]. Decentralized Identifiers (DIDs) are the fundamental component and, according to W3C's definition [33], are a type of identifier to enable decentralized digital identity. W3C defines a DID as a string composed of three parts, although exceptions exist, separated by a colon (:), with the first string being the *did* keyword, the second string representing the *DID method*, which specifies the implementation of the DID, and the third and last string being a specific *unique identifier*. The unique identifier is method specific and it may contain colons for establishing hierarchically namespaces, or for identifying instances of the verifiable data registry. As for example, the DID `did:ethr:mainnet:0xb9c5714089478a327f...` is an example of DID implemented by the method `ethr` on Ethereum mainnet for the address `0xb9c5714089478...` while the DID `did:ethr:goerli:0xb9c5714089...` is a DID provided by the same method for the Ethereum Goerli testnet. In contrast to `did:ethr` where the unique identifier is derived from a private and public key pair, there are other DID methods that allow individuals to choose the specific identifier, such as the `did:web` method [34] where the user can select a common domain name as a unique identifier.

A DID is typically owned by and refers to a *DID subject*, who is an individual or a device. Similar to individuals, a DID can be associated with additional public keys. Given a DID, a process called *DID resolution* builds a *DID document* that contains the data concerning that DID, such as the cryptographic public keys used to authenticate the DID subject, the services used to communicate with the DID subject, and other attributes. Some DID methods are based on *verifiable data registries* that record additional data about DIDs, for example, the additional public keys, and these registries support the DID resolution. Examples of verifiable data registries include distributed ledgers, decentralized file systems, distributed databases, and other forms of trusted data storage. In contrast, some DID methods, such as `did:key` [35], do not involve any verifiable data registry and the resolution generates the DID document from only the DID. DID methods based on registries are typically able to offer all the benefits provided by DID and they are more suitable for long-living DIDs whose related entity signs credentials with multiple private keys, while DID methods such as DID key work better for short-living, possibly one-time use, DIDs.

In the latter case, at the application level, DIDs do not carry any statements about the subject (except for a public key, for example in `did:key`). The high-level component of SSI called *Verifiable Credentials* (VCs) fixes this gap. Verifiable Credentials are digitally signed documents expressing statements about a DID, such as health certificates. According to the W3C, a VC [5] consists of *i)* a *header* containing the DID of the issuer, the DID of the subject, timestamps such as issuance and expiration dates; *ii)* a list of *claims*,

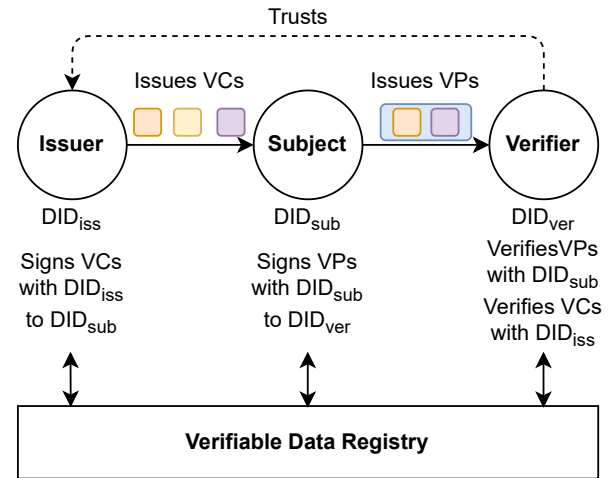


FIGURE 1. An example of SSI architecture.

i.e., application level attributes about the subject; *iii)* a *proof*, i.e., the signature and the signature-related data (such as the algorithm used). The typical model involves an *issuer* issuing credentials to *subjects* who, in turn, show them to *verifiers* encapsulated in a *Verifiable Presentation* (VP). A presentation has the same structure as a credential, but it carries a list of credentials instead of a list of claims. The verifier accepts a presentation and its credentials if: *i)* the credentials signatures validate correctly; *ii)* the credentials are signed by trusted issuers; *iii)* the recipient of each credential, i.e., the subject's DID in the credential, matches the DID of the subject who sent the presentation; *iv)* the presentation's signature validates correctly; *v)* the credentials are not expired nor revoked. Figure 1 shows an issuer, identified with DID_{iss} , issuing 3 credentials signed with DID_{iss} to a subject, identified with DID_{sub} . The subject shows only 2 credentials to a verifier by storing them in a presentation (represented by the blue box in the figure) signed with DID_{sub} . The verifier verifies the signature of the presentation with DID_{sub} , it checks the credentials are signed by a trusted issuer and verifies the signatures with DID_{iss} , and it checks the recipient of each credential matches the sender of the presentation. The verification of the signatures involves a DID resolution process from the verifier to retrieve the public keys of DID_{sub} and DID_{iss} , optionally querying a registry (which depends on the DID method).

B. MERKLE TREE AND MERKLE PATRICIA TRIE

Cryptographic data structures, such as hash trees, are extensively used in blockchains to verify data. For example, Bitcoin and Ethereum implement Merkle Trees and Merkle Patricia Tries to detect tampering attempts to transactions and to implement a key-value storage respectively.

A Merkle Tree [36] is a binary hash tree built on a sequence of data blocks $d_1, \dots, d_n$ (see Figure 2). It consists of n leaves and each leaf l_i contains the hash value $H(d_i)$ of the block d_i .

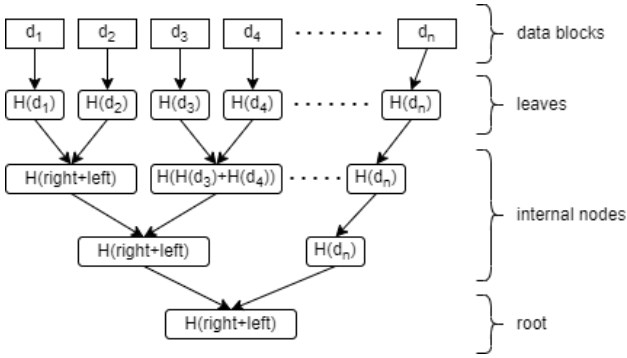


FIGURE 2. Construction of a Merkle Tree

Internal nodes v of a Merkle tree have at most two children nodes ($v.left$ and $v.right$) and only one father node ($v.father$). Each internal node v and the root node contains the hash of the concatenation of the hash values associated to the children nodes, i.e., $H(v.right + v.left)$. The root of a Merkle tree can be used to verify that a specific block of data has been considered during the construction of the Merkle tree. For example, using Figure 2, the hash proofs required by a verifier in order to link the data block d_2 to the root of the Merkle tree are $H(d_1)$, $H(H(d_3) + h(d_4))$, and $H(d_n)$. By using such hash proofs, the verifier is able to recompute the hash values on the path from leaf d_2 until the root of a Merkle tree. If we assume that the Merkle tree is a perfect binary tree, the time complexity for creating and verifying a proof is $\mathcal{O}(\log n)$, i.e., the height of the tree.

A Merkle Patricia Trie is a data structure first described by Ethereum in its yellow paper [37] and it merges the capabilities of Merkle Tree with those of Patricia Trie [38], providing the ability to index pairs (key , $value$). The key is used to navigate the Merkle Patricia Trie from the root node to the leaf that contains the corresponding value. The links among the nodes of the tree are represented by using *hash pointer*, i.e., each node stores a cryptographic hash of their children nodes. A node of the Merkle Patricia Trie can be:

- **NULL**: an empty node;
- **extension** [$path$, p]: a node consisting of a sequence of characters and a hash pointer;
- **branch** [$v_0, \dots, v_n$, p]: each v_i corresponds to a symbol of the alphabet and it can be either NULL or a hash pointer to a node. The last element p of the sequence can be NULL or may contain a *value*, depending on whether the node is terminal for a pair (key , $value$);
- **leaf** [$path$, $value$]: it represents a (key , $value$) pair and consists of the last character of the key and the corresponding value.

A Merkle Patricia Trie can be seen as a radix tree where edges are labeled with strings of arbitrary length derived from key prefixes. To verify that a certain (key , $value$) pair does or does not exist within a Merkle Patricia Trie with a given root, the algorithm consumes the characters of the key to navigate the Trie until a path whose labels on the edges

Selective disclosure Approaches	Papers
atomic-based	[42] [9]
hash-based	[43] [44] [9] [45] [46] [47]
hash tree-based	[48] [49] [50] [51]
encryption-based	[52] [53]
signature-based	[54] [55] [56] [57] [58] [59]

TABLE 2. Classification of the approaches for selective disclosure

correspond to the *key* is found. Given such a path, anyone can provide a proof that proves the Trie includes a *value* at a specific path by providing the hashes of each node joining the path. Consequently, the time complexity for insertions and lookups is $\mathcal{O}(\log(n))$, where n is the number of symbols of the alphabet [37].

C. BBS SIGNATURE SCHEME

The original definition of the BBS signature scheme is authored by Boneh, Boyen, and Shacham (BBS) [39] and, with respect to other schemes, it grants additional properties, such as selective disclosure, unlinkable proofs, and proof of possession [40]. Taking the actors of Figure 1 as a reference, the BBS scheme allows the issuer to sign a group of messages, producing a single and constant-sized signature. The subject, given the group of messages and the related signature from the issuer, in order to disclose only a subset of these messages, can generate a proof from such a subset that, if correctly validated by the verifier, guarantees the integrity and authenticity of the revealed messages while hiding the others from the verifiers. This grants zero-knowledge properties to the proof since it hides the values of the undisclosed messages, thus achieving selective disclosure. Plus, it is impossible, or very difficult, for the verifier to understand if two proofs received by the same subject are computed from the same signature. Signatures are constructed from pairing-friendly elliptic curves, such as Barreto-Lynn-Scott (BLS) curves [41].

IV. A CLASSIFICATION OF SELECTIVE DISCLOSURE STRATEGIES

This section describes the existing approaches proposed to resolve the problem of selective disclosure in SSI by extending and refining the classification already proposed in [32]. In the last years, researchers have made a great effort to design efficient and secure protocols that allow selective disclosure of Verifiable Credentials using different types of approaches. The current literature mainly discusses how each approach achieves selective disclosure of Verifiable Credentials and formally proves its security properties [60]. However, the characteristics of the proposed solutions are very different from each other and the trade-offs between the performance, functionality, and usability of the approaches remains unexplored. The analysis of the related work revealed that the current solutions for selective disclosure of Verifiable Credentials can be classified into five different approaches, which are summarized in Table 2.

The *atomic-based* approach is a simple approach for providing selective disclosure, where the issuer produces Verifiable Credentials containing exactly one claim, and the subject can selectively disclose only the claims they want by inserting only the corresponding credentials in a Verifiable Presentation. The approach has been suggested as a solution by the W3C in the Verifiable Credentials implementation guidelines [42] and formalized in [9].

The *hash-based* approach exploits hash functions to selectively reveal different parts of a credential and it has been used in several application domains, in combination with the blockchain, to protect the confidentiality of credentials while making them verifiable [43], [44]. A version of this approach (named SD-JWT [61]) is currently in the process of being standardized by IETF and it is implemented by using JSON Web Token (JWT). In particular, the issuer stores in a credential the hash values of its claims, thus hiding them, and shares the plain text values of such claims with the subject separately; the subject stores in the presentation the credential and the claims they want to share with a verifier; the verifier re-applies the hash on the claims to check if the result matches the hash values in the Verifiable Credential. Authors of [9], [46] exploit hash-based message authentication code in order to selectively disclose the credentials' attributes to a verifier. In particular, the hash-based message authentication code takes as input a hash function, a secret value, and a message, and combines these elements to return a transformation of the message.

The problem of selective disclosure has been investigated also in trust negotiations [45], [47], i.e., a protocol engaged by two parties to establish trust by requesting and disclosing credentials' attributes. In particular, they propose to use hash-commitment technique where the holder of credential generates a random secret string r for each attribute of the credential, concatenates each attribute's value with the corresponding secret r , computes the hash of the concatenation and sends it to the issuer to be verified and signed.

The *hash tree-based* approach exploits cryptographic hash data structure to keep secret and verifiable all the claims of the credentials. For example, Saito and Watanabe [48] propose a method to selectively hide attributes in an XML document associating with every attribute a nonce and utilizing Merkle proofs, with the tree root stored on a blockchain, to prove the existence of such attribute in the document, but they do not provide any measurements. Mukta et al. [49] implement the selective disclosure of a credential constructing a Merkle tree where leaves are the claims values and each node of the tree is hashed with a nonce generated by Goldreich-Goldwasser-Micali pseudo-random functions. Upon receiving the list of nonces, hashed claims, and clear claims, a verifier generates the same Merkle tree and compares the root with the one signed by the issuer to validate the authenticity of the disclosed claims. The generation of the tree, performed while issuing and verifying a credential, takes between 2 and 8 ms for 4 to 128 claims.

Authors of [50] propose a blockchain-based degree verifi-

cation system where each leaf node of a Merkle tree represents a hashed degree certificate of a student. The university sends a transaction consisting of the tree Merkle root to the blockchain and provides the student with a physical degree certificate consisting of a QR code that is used to validate degree information. The QR code embeds the certificate degree information, the block number and identifier of the transaction containing the tree Merkle root, and the specific nodes of the Merkle tree which are used to verify that the degree certificate belongs to the tree.

Another similar approach is proposed in [51], where the issuer computes the erasure code on each claim, encrypts them with the public key of the holder, and considers them as leaves of a Merkle B-tree (MBTree). The hash value of the root is signed by the issuer and stored on the blockchain while the encrypted claims are stored Off-chain on a Distributed Hash Table (DHT). The holder of the credential computes a merkel proof for each claim to be revealed and sends to the verifier a VP consisting of the decrypted claim's value and the corresponding proof.

The *encryption-based* approach is another popular solution for selective disclosure, which is similar to the hash-based approach, but it exploits cryptographic algorithms to hide the values of the claims in Verifiable Credentials. The issuer encrypts the claims of the credentials they produce by using secret keys which are shared with the subject. In turn, the verifier decrypts only the ciphertext of the claims for which they receive the corresponding keys from the subject. The solutions existing in the literature differ from each other by the type of encryption algorithms used to protect claims of the credentials. For example, authors of [52] exploit attribute-based encryption to hide claims' values based on attributes of the users. In contrast, Sonnino et al. [53] propose a solution for permissioned blockchains based on threshold cryptography. Such solution allows a subject to aggregate the parts of a credential that they receive from different authorities and selectively disclose them with a publicly verifiable protocol.

Finally, the *signature-based* approach includes various preliminary solutions that aim to define signature schemes that natively support the selective disclosure of claims. As part of this approach, Hyperledger Indy² was one of the first frameworks supporting selective disclosure of Verifiable Credentials. In this approach, the RSA based Camenisch-Lysyanskaya signature [54] is used by the issuer to create a set of public/private key pairs, one per claim in the credential. The corresponding private keys will be used to sign the claims when credentials to be issued are created. The Hyperledger Indy community is currently discussing an enhanced version of the original approach based on BBS+ signature [55], which has smaller key sizes, and therefore shorter execution times for generating keys, signature, and for creating proofs³.

²<https://indy.readthedocs.io/> [Accessed on 10 November 2025]

³Anonymous Credentials 2.0: <https://github.com/hyperledger-archives/ursa-docs/tree/189581b0133420df4fee9d256232f81767118152/specs/anoncreds2> [Accessed on 10 November 2025]

Narumanchi et al. [56] exploit only Boneh–Lynn–Shacham (BLS) signature where an issuer takes about 11 ms to generate a signature for each claim by hashing the value of the claim, randomizing the hash value, and then signing it by using a private key. The verification of the claims of a credential is implemented as an interactive process between the subject and the verifier. With this approach, the subject takes between 12 and 29 ms to build a proof for 1 to 5 claims, and the verifier takes about 8 ms to verify the proof for 1 to 5 claims. These times increase to about 600–700 ms considering the transaction to a Hyperledger Fabric with two peers and a certification authority. Babel and Sedlmeir [57] present a method based on zkSNARK zero-knowledge proofs to create anonymous credentials that allow subjects to also selectively disclose internal claims. The authors provide measurements computed on a laptop and on a Raspberry Pi varying the number of Rank One Constraint System (R1CS) constraints, which are used within a zkSNARK scheme to keep track of the values in a computation and expressed as vectors. Such constraints represent cryptographic functions, such as the application of Poisedon and SHA256 hashing or EdDSA and ECDAS signatures, and the proof generation time raises as the number of constraints increases.

The authors of [58] extend the LDP-BBS+ scheme proposed in [40] in order to make it more fine-grained.

A solution that hide the identity of the issuer of the credential to both the holder and the verifier is proposed in [59]. In particular, the proposed solution is based on mercurial signatures: a cryptographic scheme that allows to transform a signature into another equivalent signature.

Given the existence of various methods to implement such approaches, it is necessary to understand the resources required by each party to execute them. To this aim, we identify the following metrics: the time the issuer needs to create Verifiable Credentials so that its claims can be selectively disclosed; the time the subject needs to select the claims and create the Verifiable Presentation; the time the verifier needs to verify the credential and presentation; the size of the credentials, presentations, and any support data structure. Therefore, given two or more methods that have comparable qualitative benefits, such as security and privacy, the decision can be directed toward the method that is less expensive resource-wise.

V. PRELIMINARIES AND BASIC ELEMENTS

In this section, we describe the basic elements of a generalized SSI system introducing the symbols, summarized in Table 3, and a reference example that will be used throughout the paper. As shown in Figure 1, let *issuer* be an entity in control of DID_{iss} , which is trusted by a number of *verifiers* to issue some specific types of Verifiable Credentials. For instance, the University of Pisa is an issuer of Verifiable Credentials certifying the degrees of its students. The University of Pisa is trusted by all the other universities, companies, and public institutions as issuers of valuable degree certificates (bachelor, master,

Symbol	Description
DID_{name}	A DID controlled by <i>name</i>
VC	A Verifiable Credential $\langle C_{VC}, M_{VC}, P_{VC} \rangle$
VP	A Verifiable Presentation $\langle VC, M_{VP}, P_{VP} \rangle$
C_{VC}	Set of claims in a VC $\{claim_i : val_i\}$
$claim_i, val_i$	The i_{th} claim with name $claim_i$ and value val_i
$M_{VC/VP}$	The metadata in VC or VP
$P_{VC/VP}$	The verifiable proof of VC or VP

TABLE 3. Symbols used in this paper.

doctoral). Municipalities are another example of issuers who release credentials whose claims concern personal data such as the date of birth, the city of birth, the nationality, the city and the address of residence of a subject, and work institutions issuers who release credentials to its employees. We model a Verifiable Credential VC as a statement consisting of a triple:

$$VC = \langle C_{VC}, M_{VC}, P_{VC} \rangle \quad (1)$$

where C_{VC} is the set of claims about a *subject*, M_{VC} is the set of metadata of the credential (e.g., it contains the issuer's DID, DID_{iss} , the credential type, the credential id, the issuance and expiration dates, and the subject's DID, DID_{sub}), and P_{VC} is the proof generated by the issuer with a digital signature. The claims in the set C_{VC} are assertions in the form of attribute name-value pairs:

$$C_{VC} = \{claim_1 : val_1, \dots, claim_n : val_n\} \quad (2)$$

Each val_i could either be a simple or a complex and structured value, such as a dictionary or an array. To preserve the focus of the paper and to keep the addressed research problem within a manageable framework, we are confining our analysis to simple values, since complex types can be unwrapped and managed as simple ones.

The following shows a simplified Verifiable Credential in JSON format, representing a degree that a university issues to certify a graduated student. Its claims C_{VC} may include the degree course, the degree grade, the normal duration of the course in years and the number of years taken by the student, and the graduation date if it differs from the date of issuance of the credential.

```
VC: {
  Mvc: {
    "@context": ["https://www.w3.org/..."],
    "type": ["VerifiableCredential"],
    "issuer": DIDiss,
    "subject": DIDsub,
    "issuance": 2023-02-24,
    "expiration": None
  },
  Cvc: {
    "course": "Computer Science",
    "degree": "Bachelor degree",
    "yearsDuration": 3,
    "yearsEmployed": 5,
    "grade": 110,
    "laude": true,
    "graduationDate": 2023-02-21
  }
}
```



```

    },
    Pvc: {
      "type": "RsaSignature2018",
      "signature": "jHey...AAMP"
    }
  }

```

Listing 1. Simplified data model of a Verifiable Credential

A Verifiable Presentation VP can be created by the subject in order to securely show a *verifier* a subset of the Verifiable Credentials they received from several issuers. We model a VP as a triple:

$$VP = \langle VC[], M_{VP}, P_{VP} \rangle \quad (3)$$

where $VC[]$ is the list of Verifiable Credentials included in the presentation, M_{VP} is the set of metadata of the presentation (e.g., it contains DID_{sub} , the presentation issuance and/or expiration dates), and P_{VP} is the proof used to verify the presentation. Although a presentation may contain more than one credential, the model used in this paper considers a presentation consisting of only a single credential. However, the methods defined in the following section can be applied to all the credentials the subject wants to place in their presentation.

To preserve their privacy, a subject may not want to disclose all the claims included in the VC to show to the verifier. For example, with reference to the previous example of VC representing a degree issued by the university, the subject could not want to show that they took 5 years to complete a 3 years bachelor's course. Plus, the verifier may not need to know all the values as well because only a subset of them could be sufficient for making a decision. However, being the VC signed by the issuer, the subject cannot hide or remove some claims from that credential without invalidating the signature, and therefore the values of all the claims will be disclosed to the verifier. To address this issue, selective disclosure solutions have been introduced.

VI. SELECTIVE DISCLOSURE METHODS

The task of hiding or revealing parts of a credential is known as *selective disclosure* and, as shown in Section IV, the currently known solutions are based on different approaches. This paper extends a previous research investigation on selective disclosure methods [9] presenting, analyzing, and comparing some instances of the five approaches previously described: atomic-based, hash-based, encryption-based, hash tree-based, and signature-based.

As concern the Atomic-based approach, we adopted the method recommended by the W3C in the Verifiable Credentials implementation guidelines as our reference implementation [42]. For the hash-based approach, we consider the method SD-JWT proposed by [61], while for the encryption-based approach we adapted the solution proposed in [52]. The implementation of the hash tree-based approach takes inspiration from [48], which allows proving the existence of attributes in XML documents. Instead, the reference implementation used by the signature-based approach is based

on the principles outlined in the Verifiable Credentials Implementation Guidelines [42], and it utilizes zero-knowledge proof based on BBS+ signature scheme.

In this section, we describe the underlying methodology of each approach. Each approach is presented in 3 phases, each of which is executed by one of the stakeholders of the SSI scenario: 1) *Issuer: Preparing the credentials*: describes the steps executed by the issuer to create the credentials to be sent to the subject; 2) *Subject: Selective disclosure*: describes how the subject conceals the claims while preparing the Verifiable Presentation; 3) *Verifier: Credential verification*: describes the steps the verifier performs upon receiving the presentation to check the authenticity and to extract the information. We provide the time and space requirements related to each phase, assuming that the issuer sends n claims to the subject who, in turn, discloses $k \leq n$ claims to the verifier. Table 4 groups these requirements together to illustrate the differences between the approaches in each phase. The implementation of the algorithms are described in Section VII, they are quantitatively compared in Section VIII, and their usability and performance are discussed in Section IX.

In order to perform a fair comparison, all the approaches we analyzed utilize common helper functions: the *sign* function is a signature function (e.g., ES256K) configured to sign data with the private key generating a DID, for example, DID_{iss} and DID_{sub} ; the *verifyVC()* and *verifyVP()* perform the DID resolution (see Section III) and verify the signature on a VC and VP respectively. On the verifier's end, in the following description we omit the operations common to all the algorithms, such as checking the presence of DID_{iss} in the verifier's trusted issuers list or checking that DID_{sub} in a Verifiable Credential matches the issuer of the Verifiable Presentation. Finally, we assume the data is transferred between the actors by means of a secure channel that guarantees confidentiality and integrity.

A. ATOMIC-BASED

As declared by W3C in the Verifiable Credentials implementation guidelines [42], the naive approach to achieve selective disclosure of a set of n claims is using n atomic credentials with one claim each, instead of a single credential including all the claims. Therefore, the subject can select the subset of claims to include in their presentation by selecting the corresponding credentials. The functions used in this approach (*createCredential*, *createPresentation*, and *verifyPresentation*) are reported in Algorithm 1, while the VCs and VP production workflow is depicted in Figure 3.

1) Issuer: Preparing the credentials. Given a set of claims C_{VC} , the issuer prepares as many VCs as the number of claims in C_{VC} . The issuer executes the *createCredential* function passing as input parameters, the set of claims, the metadata M_{VC} (e.g., the expiring date) along with some approach specific metadata, and the signature function *sign* (the details of the *sign* function are given at the beginning of this section). For each claim in C_{VC} (line 3), the algorithm creates a Verifiable Credential containing only one claim, with the

Operations					
	Atomic	Hash	Encryption	Tree	BBS
Issuer	n signatures	n HMAC + 1 signature	$n SK_E$ + 1 signature	$2n$ HMAC + n inclusion proof gen + 1 signature	1 BBS signature + 1 signature
Subject	k copies + 1 signature	$(k + 1)$ copies + 1 signature	$(k + 1)$ copies + 1 signature	$(k + 1)$ copies + 1 signature	1 BBS proof gen + $(k + 3)$ copies + 1 signature
Verifier	$k + 1$ signature verifications	k HMAC + 2 signature verifications	$k SK_D$ + 2 signature verifications	k inclusion proof verification + 2 signature verifications	1 BBS proof verification + 2 signature verifications

Space					
	Atomic	Hash	Encryption	Tree	BBS
VC	n VCs with 1 value each	n hash values	n encrypted values	1 root (hash value)	1 BBS signature + 1 BLS public key
VP	k VCs	1 VC	1 VC	1 VC	1 VC + 1 BBS proof + 1 nonce
Attributes	-	k tuples (path, key, value)	k tuples (path, key, nonce)	k tuples (proof, value, value)	k values

TABLE 4. Operations performed by each approach (table above) and space required by the main data structures involved (table below).

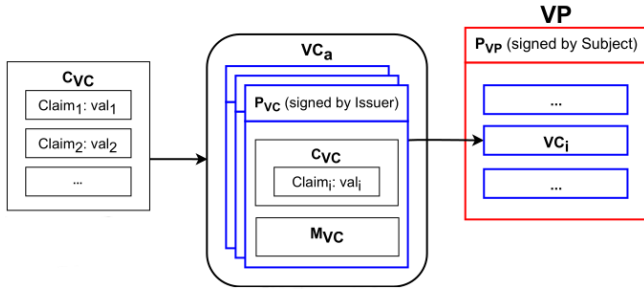


FIGURE 3. Illustration of the atomic credentials selective disclosure. The plain arrows represent a copy operation. A blue box represents a signed VC, a red box a signed VP.

value represented in clear, and the metadata. The credential is signed and inserted in a set VC_a . In the end, the whole set VC_a is sent to the subject. This step requires creating and signing n VCs.

2) Subject: Selective disclosure. The subject who received the set VC_a discloses to a verifier a subset of k VCs by creating a VP invoking the *createPresentation* function. The function takes as input parameters the set (VC_a), the metadata of the presentation (M_{VP}), the function used for signing the presentation (*sign*), and the indices of the claims to be disclosed (*disclosedAttr*). The claims that a subject wants to share are retrieved from VC_a and included in the set of Verifiable Credentials $VC_d \subseteq VC_a$ to be embedded in the VP (line 13). Finally, the algorithm signs the set VC_d along with the metadata of the presentation in order to create a proof P_{VP} for the presentation and returns the presentation VP_a to the subject. This step requires copying k VCs into a VP and signing the VP. The VP size is proportional to k times the size of a VC plus the size of the signature.

3) Verifier: Credential verification. The verification of an atomic Verifiable Presentation by a verifier is performed by the *verifyPresentation* function that invokes *verifyVP* (line 22) resolving DID_{sub} (see the beginning of this section). The

Algorithm 1: Functions of the atomic-based approach

```

1 def createCredential( $C_{VC}$ : set,  $M_{VC}$ : set, sign: fun):
2    $VC_a = \{\}$ 
3   for ( $claim_i$ :  $val_i$ )  $\in C_{VC}$  do
4      $C_{VC}^i = (claim_i : val_i)$ 
5      $M_{VC}^i = M_{VC} \cup \{claim\ metadata\}$ 
6      $P_{VC}^i = createProof(C_{VC}^i, M_{VC}^i, sign)$ 
7      $VC_a \leftarrow \langle C_{VC}^i, M_{VC}^i, P_{VC}^i \rangle$ 
8   end
9   return  $VC_a$ 
10
11 def createPresentation( $VC_a$ : set,  $M_{VP}$ : set, sign: fun,
12   disclosedAttr: set):
13    $VC_d = \{\}$ 
14   for  $i \in disclosedAttr$  do
15      $\langle C_{VC}^i, M_{VC}^i, P_{VC}^i \rangle = VC_a[i]$ 
16      $VC_d \leftarrow \langle C_{VC}^i, M_{VC}^i, P_{VC}^i \rangle$ 
17   end
18    $P_{VP} = createProof(VC_d, M_{VP}, sign)$ 
19    $VP_a = \langle VC_d, M_{VP}, P_{VP} \rangle$ 
20   return  $VP_a$ 
21
22 def verifyPresentation( $VP_a$ :  $\langle VC_d, M_{VP}, P_{VP} \rangle$ ):
23   return verifyVP( $VP_a$ )
24
25 def verifyCredential( $VC_a$ : set):
26   for  $\langle C_{VC}^i, M_{VC}^i, P_{VC}^i \rangle \in VC_a$  do
27     if verifyVC( $\langle C_{VC}^i, M_{VC}^i, P_{VC}^i \rangle$ ) == false then
28       return failure
29     end
30   end
31   return success

```

verification of the inner credentials is performed by the *verifyCredential* function (line 24) that invokes *verifyVC* resolving DID_{iss} (see the end of Section VI). This step requires the verification of $k + 1$ signatures: one signature per disclosed VC, and the VP signature.

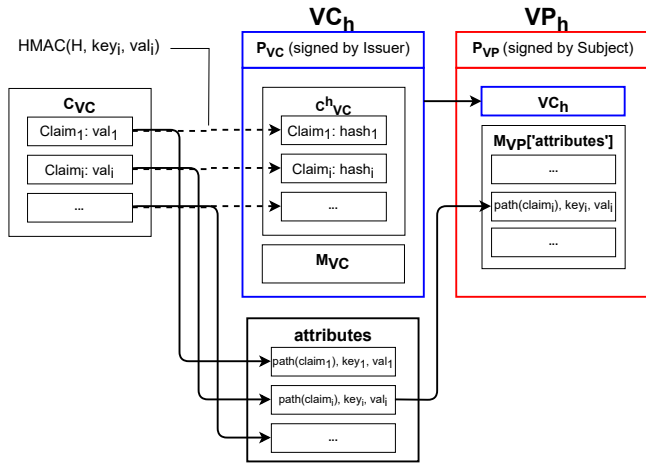


FIGURE 4. Illustration of the hash-based selective disclosure. Plain arrows represent copy operations and dashed arrows results in a computation. A blue box represents a signed VC, a red box a signed VP.

B. HASH-BASED

In the hash-based selective disclosure approach [61], the issuer hides the values of the claims by hashing them and storing in the Verifiable Credential only the related digests. The plain text values of the claims are sent separately to the subject. The subject creates a Verifiable Presentation containing the Verifiable Credential embedding all the digests, and a data structure storing the plain values of only the claims they want to reveal. The verifier checks if the hash of the received claims matches the digests in the credential that, being signed by the issuer, proves their integrity. This approach can be easily customized and integrated into existing systems, including the ones running on smartphones or low-power devices [62]. Indeed, popular hash functions are based on solid and audited standards, not computationally intensive, and widely utilized. The approach presented in this section utilizes a Hash-based Message Authentication Code $HMAC(H, key, m)$ [9], [63]:

$$HMAC(H, key, m) = H(key \oplus opad) \parallel H((key \oplus ipad) \parallel m)$$

where H is a cryptographic hash function, key a random key (or nonce), m the data to hash, symbol $\oplus$ denotes bitwise XOR, symbol $\parallel$ denotes concatenation, the constants opad and ipad are the outer and the inner padding values, respectively. We notice that the $HMAC$ function invokes the cryptographic hash function H twice during its execution. The functions used in this approach (*createCredential*, *createPresentation*, and *verifyPresentation*) are reported in Algorithm 2, while the VCs and VP production workflow is depicted in Figure 4. We decided to use $HMAC$ as a Keyed-Hash Message Authentication Code because it is a recognized standard ([63], [64]), as well as for its efficiency [65], simplicity, and provable security [66]. Nevertheless, the architecture remains flexible, and any other secure Keyed-Hash Message Authentication Code (such as KMAC [67]) could be used depending on the specific performance and requirements.

Algorithm 2: Functions of the hash-based approach

```

1 def createCredential( $C_{VC}$ : set,  $M_{VC}$ : set,  $sign$ : fun,  $H$ : fun,
   $K$ : int):
2    $C_{VC}^h = \{\}$ 
3    $attributes = \{\}$ 
4   for ( $claim_i : val_i$ )  $\in C_{VC}$  do
5     ( $key_i, hash_i$ ) = hashAttribute( $val_i, K, H$ )
6      $C_{VC}^h \leftarrow (claim_i : hash_i)$ 
7      $attributes \leftarrow (path(claim_i), val_i, key_i)$ 
8   end
9    $P_{VC} = createProof(C_{VC}^h, M_{VC}, sign)$ 
10   $VC_h = \langle C_{VC}^h, M_{VC}, P_{VC} \rangle$ 
11  return  $\{VC_h, attributes\}$ 
12
13 def hashAttribute( $val$ : string,  $K$ : int,  $H$ : fun):
14    $key = randomKey(K)$ 
15    $hash = HMAC(H, key, val)$ 
16   return  $\{key, hash\}$ 
17
18 def createPresentation( $VC_h$ :  $\langle C_{VC}^h, M_{VC}, P_{VC} \rangle$ ,  $M_{VP}$ : set,
   $sign$ : fun,  $disclosedAttr$ : set):
19   for ( $path(claim_i), val_i, key_i$ )  $\in disclosedAttr$  do
20      $M_{VP}[attributes'] \leftarrow (path(claim_i), val_i, key_i)$ 
21   end
22    $P_{VP} = createProof(VC_h, M_{VP}, sign)$ 
23    $VP_h = \langle VC_h, M_{VP}, P_{VP} \rangle$ 
24   return  $VP_h$ 
25
26 def verifyPresentation( $VP_h$ :  $\langle VC_h, M_{VP}, P_{VP} \rangle$ ,  $H$ : fun):
27   if verifyVP( $VP_h$ ) then
28     return verifyHashedAttributes( $VP_h, H$ )
29   end
30   return failure
31
32 def verifyCredential( $VC_h$ :  $\langle C_{VC}^h, M_{VC}, P_{VC} \rangle$ ):
33   return verifyVC( $VC_h$ )
34
35 def verifyHashedAttributes( $VP_h$ :  $\langle VC_h, M_{VP}, P_{VP} \rangle$ ,  $H$ :
  fun):
36    $VC_h = \langle C_{VC}^h, M_{VC}, P_{VC} \rangle$ 
37   for ( $path(claim_i), val_i, key_i$ )  $\in M_{VP}[attributes']$  do
38     if  $path(claim_i) \in C_{VC}^h$  then
39        $hash_i = C_{VC}^h[path(claim_i)]$ 
40        $reshashedVal = HMAC(H, key_i, val_i)$ 
41       if  $hash_i \neq reshapedVal$  then
42         return failure
43       end
44     end
45   end
46   return success

```

1) Issuer: Preparing the credentials. To create a credential with this approach, the issuer invokes the *createCredential* function and specifies as input the set of claims to be embedded in the Verifiable Credential C_{VC} , the metadata M_{VC} , the function $sign$, the $HMAC$ hash function H , and the length in bits K of the key generated to hash the claim values. For each pair ($claim_i : val_i$) $\in C_{VC}$ (line 4), the algorithm generates a random key, key_i , of K bits, computes $hash_i = HMAC(H, key_i, val_i)$ (line 5) and inserts the resulting hashed claim ($claim_i : hash_i$) into the set of hashed claims C_{VC}^h (line

6). Instead, key_i , val_i , and the path used to reach the claim in the credential, $path(claim_i)$, are stored in a separate data structure, named *attributes* (line 7). The algorithm generates a proof, P_{VC} , by signing the new credential consisting of the hashed claims

$$C_{VC}^h = \{claim_1 : hash_1, \dots, claim_n : hash_n\} \quad (4)$$

and the metadata M_{VC} . Finally, the algorithm builds and returns the Verifiable Credential VC_h and the *attributes* data structure:

$$VC_h = \langle C_{VC}^h, M_{VC}, P_{VC} \rangle; \quad (5)$$

$$attributes = \langle (path(claim_1), val_1, key_1), \dots, (path(claim_n), val_n, key_n) \rangle$$

This step requires generating n random keys, computing n HMAC operations, and signing the VC. The issuer sends VC_h and *attributes* to the subject. Recall, we assume a communication with a secure channel that guarantees confidentiality and integrity (see Section V), meaning that the keys in *attributes* cannot be altered or eavesdropped.

2) Subject: Selective disclosure. When the subject wants to disclose to a verifier a subset of claims $disclosedAttr \subseteq attributes$ of a VC, they create a hashed Verifiable Presentation VP_h , built from VC_h , to reveal only such claims. The subject provides as input of the *createPresentation* function the hashed Verifiable Credential VC_h , the metadata of the presentation M_{VP} , the function to be used to sign the credential (*sign*), and the subset $disclosedAttr$. At first, the function creates the *attributes* data structure in the presentation metadata (line 20). To this aim, for each claim $claim_i \in disclosedAttr$, the function inserts in the presentation metadata $M_{VP}['attributes']$ the triple $(path(claim_i), key_i, val_i)$. As a result, in this solution, the metadata contains an additional field that is not present in a normal VP, which is the *attributes* structure. As second operation, the function creates the proof P_{VP} signing VC_h and the metadata M_{VP} (line 22). Finally, the function creates the Verifiable Presentation:

$$VP_h = \langle VC_h, M_{VP}, P_{VP} \rangle \quad (6)$$

and return it to the subject. This step requires copying the VC, copying the information of the k entries from *attributes* into the VP metadata, signing the VC set along with the VP metadata, and creating a new VP data structure embedding the VC set, the VP metadata, and the signature. The size of the VP is proportional to the number of hash values in VC_h , n , plus the number of triples $(path(claim_i), key_i, val_i)$ for the disclosed fields, k . Moreover, the size of the VP also includes the size of the signature.

3) Verifier: Credential verification. The *verifyPresentation* (line 26) and the *verifyCredential* functions (line 32) verify the signature of VP_h and VC_h respectively. If the signatures are correct, the verifier checks whether the values provided for the disclosed claims match the hashes signed by the issuer. This check is implemented by the function *verifyHashedAttributes* (line 35), called by the function *verifyPresentation*.

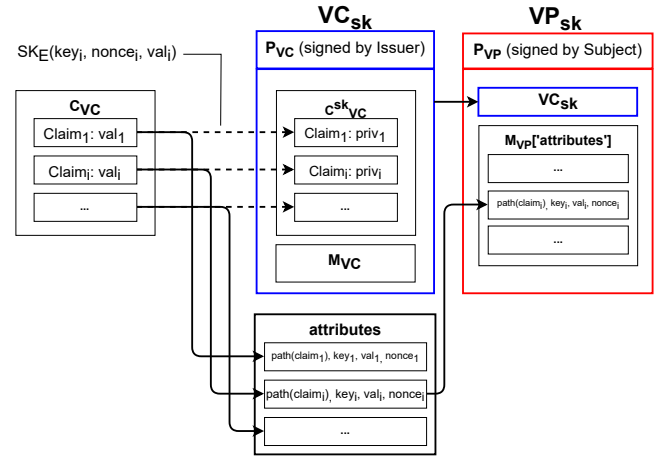


FIGURE 5. Illustration of the encryption-based selective disclosure. Plain arrows represent copy operations and dashed arrows results in a computation. A blue box represents a signed VC, a red box a signed VP.

For each triple $(path(claim_i), val_i, key_i)$ in $M_{VP}['attributes']$, the verifier: i) checks $claim_i$ belongs to C_{VC}^h to be sure the claim has been issued by the issuer (line 38); ii) retrieves the associated hash value $hash_i$ from C_{VC}^h (line 39), and iii) checks $HMAC(H, key_i, val_i)$ equals to $hash_i$ (line 41). If all the conditions are met, the verifier will process the values of the claims in the subject's request. This step requires the execution of k HMAC operations and the verification of the VC_h and VP_h signatures.

C. ENCRYPTION-BASED

This approach is very similar to the hash-based, but it utilizes an encryption algorithm, instead of hashing, to hide the values of the claims. The approach can be configured to use different types of encryption, either based on symmetric [9] or asymmetric schemas [52], [53]. However, in this paper we focus on symmetric encryption since, compared to asymmetric encryption, it is faster while its cost and computational requirements are much lower. We will denote the symmetric encryption schema SK , the encryption algorithm SK_E , and the related decryption algorithm SK_D . The *createCredential*, *createPresentation*, and *verifyPresentation* functions related to this approach are reported in Algorithm 3, while the VCs and VP production workflow is depicted in Figure 5.

1) Issuer: Preparing the credentials. The issuer of the credential specifies as input of the *createCredential* function the set of claims to be embedded in the Verifiable Credential C_{VC} , the metadata M_{VC} , the signature function *sign*, the symmetric encryption algorithm SK_E , and the length in bits K of the key used to encrypt the claim values (e.g., 128, 192 or 256 bits). For each pair $(claim_i : val_i) \in C_{VC}$ (line 3), the function generates a random symmetric key key_i of K bits, a random *nonce* _{i} of 128 bits, and uses them to encrypt the value of the claim val_i thus generating the encrypted value $priv_i$ that is stored in the set of encrypted claims C_{VC}^{sk} (line 4). Moreover, the triple $(path(claim_i), key_i, nonce_i)$, where $(path(claim_i))$ is the path of the i -th claim in the credential, is stored in the

Algorithm 3: Functions of the encryption-based approach

```

1 def createCredential( $C_{VC}$ : set,  $M_{VC}$ : set,  $sign$ : fun,  $SK_E$ : fun,
    $K$ : int):
2    $C_{VC}^{sk}, attributes = \{\}$ 
3   for ( $claim_i : val_i \in C_{VC}$  do
4     ( $key_i, priv_i, nonce_i$ ) =
       encryptAttribute( $val_i, K, SK_E$ )
5      $C_{VC}^{sk} \leftarrow (claim_i : priv_i)$ 
6      $attributes \leftarrow (path(claim_i), key_i, nonce_i)$ 
7   end
8    $P_{VC} = createProof(C_{VC}^{sk}, M_{VC}, sign)$ 
9    $VC_{sk} = \langle C_{VC}^{sk}, M_{VC}, P_{VC} \rangle$ 
10  return  $\{VC_{sk}, attributes\}$ 
11
12 def encryptAttribute( $val$ : string,  $K$ : int,  $SK_E$ : fun):
13    $key = randomKey(K)$ 
14    $nonce = randomKey(128)$ 
15    $priv = SK_E(nonce, key, val)$ 
16   return  $\{key, priv, nonce\}$ 
17
18 def createPresentation( $VC_{sk} : \langle C_{VC}^{sk}, M_{VC}, P_{VC} \rangle$ ,  $M_{VP}$ : set,
    $sign$ : fun,  $disclosedAttr$ : set):
19   for ( $path(claim_i), key_i, nonce_i \in disclosedAttr$  do
20      $M_{VP}[attributes'] \leftarrow (path(claim_i), key_i, nonce_i)$ 
21   end
22    $P_{VP} = createProof(VC_{sk}, M_{VP}, sign)$ 
23    $VP_{sk} = \langle VC_{sk}, M_{VP}, P_{VP} \rangle$ 
24   return  $VP_{sk}$ 
25
26 def verifyPresentation( $VP_{sk} : \langle VC_{sk}, M_{VP}, P_{VP} \rangle$ ,  $SK_D$ :
   fun):
27   if verifyVP( $VP_{sk}$ ) then
28     return verifyEncryptedAttributes( $VP_{sk}, SK_D$ )
29   end
30   return failure
31
32 def verifyCredential( $VC_{sk} : \langle C_{VC}^{sk}, M_{VC}, P_{VC} \rangle$ ):
33   return verifyVC( $VC_{sk}$ )
34
35 def verifyEncryptedAttributes( $VP_{sk} : \langle VC_{sk}, M_{VP}, P_{VP} \rangle$ ,
    $SK_D$ : fun):
36    $\langle C_{VC}^{sk}, M_{VC}, P_{VC} \rangle = VC_{sk}$ 
37   for ( $path(claim_i), key_i, nonce_i \in M_{VP}[attributes']$  do
38     if  $path(claim_i) \in C_{VC}^{sk}$  then
39        $priv_i = C_{VC}^{sk}[path(claim_i)]$ 
40        $val_i = SK_D(nonce_i, key_i, priv_i)$ 
41       if  $val_i == null$  then
42         return failure
43       end
44     end
45   end
46   return success

```

$attributes$ data structure (line 6). As last operation, the function *createCredential* generates a proof, P_{VC} , by signing the new credential consisting of the encrypted claims

$$C_{VC}^{sk} = \{claim_1 : priv_1, \dots, claim_n : priv_n\} \quad (7)$$

and the metadata M_{VC} . Similarly to the hash-based approach, the fields that will be used by the subject to reveal the encrypted values, i.e., the random symmetric key key_i and the

random value $nonce_i$, have been stored in the data structure named *attributes*. Therefore, VC_{sk} and *attributes* are structured as follows:

$$VC_{sk} = \langle C_{VC}^{sk}, M_{VC}, P_{VC} \rangle$$

$$attributes = \{(path(claim_1), key_1, nonce_1), \dots, (path(claim_n), key_n, nonce_n)\} \quad (8)$$

The issuer sends VC_{sk} and *attributes* to the subject. This step requires generating n random keys and n random nonces, computing the SK_E encryption operation n times, and signing the VC.

2) Subject: Selective disclosure. A subject receives from the issuer a Verifiable Credential with encrypted claim values VC_{sk} along with the *attributes* set. The subject wants to disclose to a verifier a subset of claims $disclosedAttr \subseteq attributes$ by creating a Verifiable Presentation VP_{sk} . The subject provides as input of the *createPresentation* function (line 18) the encrypted Verifiable Credential VC_{sk} , the metadata of the presentation M_{VP} , the function *sign*, and the subset *disclosedAttr*. For each claim $claim_i \in disclosedAttr$ (line 19), the algorithm includes in the presentation metadata $M_{VP}[attributes']$ the triple $(path(claim_i), key_i, nonce_i)$ (line 20). As a second operation, the function creates the proof P_{VP} signing VC_{sk} and the metadata M_{VP} . Finally, the function creates the Verifiable Presentation VP_{sk} and returns it to the subject:

$$VP_{sk} = \langle VC_{sk}, M_{VP}, P_{VP} \rangle \quad (9)$$

This step requires copying the VC_{sk} and the information related to the k disclosed attributes into the *attributes* structure of the VP metadata, signing VC_{sk} and M_{VP} , and creating a new VP data structure embedding the encrypted credential VC_{sk} , the VP metadata M_{VP} , and the signature P_{VP} . The size of the VP is proportional to n encrypted values, plus k disclosure data for the disclosed fields, i.e., the triples $(path(claim_i), key_i, nonce_i)$, and the signature.

3) Verifier: Credential verification. The *verifyPresentation* (line 26) and the *verifyCredential* functions (line 32) verify the signature of VP_{sk} and VC_{sk} respectively. If the signature is successfully validated (line 27), for each triple $(path(claim_i), key_i, nonce_i)$ in $M_{VP}[attributes']$, the algorithm: i) checks $claim_i$ belongs to C_{VC}^{sk} to be sure the claim has been issued by the issuer (line 38); ii) retrieves the associated encrypted value $priv_i$ from C_{VC}^{sk} (line 39), and iii) decrypts it with $SK_D(nonce_i, key_i, priv_i)$ (line 40). The verifier approves the subject's request only after all the previous operations have been completed successfully.

This step requires k decryption operations and the verification of the VC_{sk} and VP_{sk} signatures.

D. HASH TREE-BASED

The two former approaches, based on hashing and encryption, require the Verifiable Credentials to embed all the hashed or encrypted claims and, consequently, to store them in the presentations, which may lead to unnecessary storage overhead. The following approach is based on hash trees [36],

Algorithm 4: Functions of the hash tree-based approach

```

1 def createCredential( $C_{VC}$ : set,  $M_{VC}$ : set,  $sign$ : fun,  $H_I$ : fun,
    $H_I$ : fun,  $K$ : int):
2   attributes = {}
3   Tree t=new Tree( $H_I$ )
4   for ( $claim_i$  :  $val_i$ )  $\in C_{VC}$  do
5      $nonce_i$ =randomKey( $K$ )
6     if MT then Merkle Tree
7        $hashedClaim_i$ =HMAC( $H_I$ ,  $nonce_i$ ,  $claim_i$ : $val_i$ )
8       t.put( $hashedClaim_i$ )
9     else if MPT then Merkle Patricia Trie
10       $hashedVal_i$ =HMAC( $H_I$ ,  $nonce_i$ ,  $val_i$ )
11      t.put( $claim_i$ ,  $hashedVal_i$ )
12    end
13  end
14  for ( $claim_i$  :  $val_i$ )  $\in C_{VC}$  do
15    if MT then
16       $proof_i$  = t.getProof( $hashedClaim_i$ )
17    else if MPT then
18       $proof_i$  = t.getInclusionProof( $claim_i$ )
19    end
20    attributes  $\leftarrow$  ( $claim_i$ ,  $val_i$ ,  $nonce_i$ ,  $proof_i$ )
21  end
22   $C_{VC}^{tr}$ =(root : t.root)
23   $P_{VC}$  = createProof( $C_{VC}^{tr}$ ,  $M_{VC}$ ,  $sign$ )
24   $VC_{tr}$  =<  $C_{VC}^{tr}$ ,  $M_{VC}$ ,  $P_{VC}$  >
25  return { $VC_{tr}$ , attributes}
26
27 def createPresentation( $VC_{tr}$ : <  $C_{VC}^{tr}$ ,  $M_{VC}$ ,  $P_{VC}$  >,  $M_{VP}$ : set,
    $sign$ : fun, disclosedAttr: set):
28   for ( $claim_i$ ,  $val_i$ ,  $nonce_i$ ,  $proof_i$ )  $\in$  disclosedAttr do
29      $M_{VP}$ ['attributes']  $\leftarrow$  ( $claim_i$ ,  $val_i$ ,  $nonce_i$ ,  $proof_i$ )
30   end
31    $P_{VP}$  = createProof( $VC_{tr}$ ,  $M_{VP}$ ,  $sign$ )
32    $VP_{tr}$  =<  $VC_{tr}$ ,  $M_{VP}$ ,  $P_{VP}$  >
33   return  $VP_{tr}$ 

```

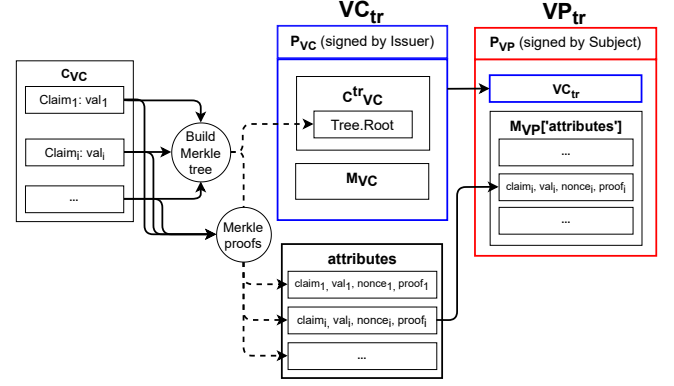


FIGURE 6. Illustration of the Merkle tree-based selective disclosure. Plain arrows represent copy operations and dashed arrows results in a computation. A blue box represents a signed VC, a red box a signed VP.

[37], storing in the Verifiable Credential, signed by the issuer, only the root of the tree and disclosing a claim with a proof of membership, i.e., a path that proves that a leaf belongs to such root. Hash trees are cryptographic data structures used to verify the integrity of portions of data (see Section III). To better analyze the capabilities of different data structures, in the following we specify the operations necessary to implement the selective disclosure of Verifiable Credentials with both Merkle Tree and Merkle Patricia Trie. Merkle Trees and Merkle Patricia Tries are chosen for their ability to implement cryptographic commitment schemes where claims corresponding to leaf nodes may be revealed and proven to be part of the verified commitment, and they are widely used in blockchain technology, e.g., in Bitcoin [68], Ethereum [37], and in distributed systems, e.g., in Bitorrent [69], and in the ZFS file system [70]. The functions used in this approach are reported in Algorithms 4 and 5, while the VCs and VP production workflow is depicted in Figure 6.

1) Issuer: Preparing the credentials. The issuer specifies as input of the *createCredential* function in Algorithm 4 the claims C_{VC} and the metadata M_{VC} of the credential, the credential signing function *sign*, the hash function used on the

leaves (H_I) and on the internal nodes of the tree (H_T), and the size K in bits of the pseudo-random nonce used to hide hash patterns in the case of claims with the same values. Using two distinct hash functions for leaves and internal nodes of the tree protects from second pre-image attacks. A tree is initialized by passing the hash function to use to compute the hash of its nodes (H_I in line 3). For each pair ($claim_i$: val_i) in C_{VC} , the algorithm creates a nonce ($nonce_i$) (line 5 of Algorithm 4) and exploits it in the H_I hash function to create the corresponding leaf of the tree. In the case of Merkle Trees (line 7), each leaf is computed by applying H_I with $nonce_i$ to the pair ($claim_i$: val_i). On the other hand, in the case of Merkle Patricia Tries (line 9), each leaf is computed by applying H_I with $nonce_i$ to the claim value val_i while the claim's name $claim_i$ is used as the key of the leaf. The *put* function (invoked in line 8 in the case of Merkle Tree and in line 11 in the case of Merkle Patricia Tries), is responsible for inserting the previously created leaf into the tree. In particular, for each new leaf inserted, the *put* function properly updates the hash values of the ancestor nodes (exploiting the H_I hash function) according to the method described in Section III-B. After the tree has been created, the algorithm builds for each pair ($claim_i$: val_i) in C_{VC} the corresponding inclusion proof, $proof_i$, by visiting the tree and stores the tuple ($claim_i$, val_i , $nonce_i$, $proof_i$) in a separate data structure named *attributes* (line 15-20). The set of claims C_{VC}^{tr} consists only of the root of the tree (line 22). C_{VC}^{tr} and the credential metadata M_{VC} are signed with the *sign* function to generate P_{VC} (line 23). Therefore, VC_{tr} and *attributes* are as follows:

$$VC_{tr} = \langle C_{VC}^{tr}, M_{VC}, P_{VC} \rangle$$

$$attributes = \{ (claim_1, val_1, nonce_1, proof_1), \dots, (claim_n, val_n, nonce_n, proof_n) \} \quad (10)$$

Finally, VC_{tr} and *attributes* are sent to the subject. This step requires the generation of n random nonces, the computation of $2n$ HMAC operations to build a tree (assuming a perfectly balanced tree), the production of n proofs that require $\log(n)$ operations each, and the signature of the credential.

Algorithm 5: Verification of hash tree-based credentials and presentations

```

1 def verifyCredential( $VC_{tr} : < C_{VC}^r, M_{VC}, P_{VC} >$ ):
2   | return verifyVC( $VC_{tr}$ )
3
4 def verifyPresentation( $VP_{tr} : < VC_{tr}, M_{VP}, P_{VP} >$ ):
5   | if verifyVP( $VP_{tr}$ ) then
6     | return verifyTreeAttributes( $VP_{tr}$ )
7   | end
8   | return failure
9
10 def verifyTreeAttributes( $VP_{tr} : < VC_{tr}, M_{VP}, P_{VP} >$ ):
11   |  $< C_{VC}^r, M_{VC}, P_{VC} > = VC_{tr}$ 
12   |  $root = C_{VC}^r[ 'root' ]$ 
13   | for  $(claim_i, val_i, nonce_i, proof_i) \in M_{VP}[ 'attributes' ]$  do
14     | if MT then Merkle Tree
15       |  $hc_i = HMAC(H_l, nonce_i, claim_i : val_i)$ 
16       |  $MT.verifyProof(root, hc_i, proof_i)$ 
17     | else if MPT then Merkle Patricia Trie
18       |  $val = MPT.verifyProof(root, claim_i, proof_i)$ 
19       | if  $HMAC(H_l, nonce_i, val_i) \neq val$  then
20         | return failure
21       | end
22     | end
23   | end
24   | return success

```

2) Subject: Selective disclosure. A subject receives from the issuer a Verifiable Credential VC_{tr} along with the attributes set. The subject can disclose to a verifier a subset of k claims $disclosedAttr \subseteq attributes$. This step works similarly to the previous approaches. As a matter of fact, the *createPresentation* function (line 27) adds the tuples $(claim_i, val_i, nonce_i, proof_i)$ associated with the claims in *disclosedAttr* to the metadata M_{VP} (lines 28-30) and creates a presentation VP_{tr} (line 32) that contains VC_{tr} , the metadata M_{VP} , and the proof P_{VP} obtained by signing VC_{tr} and the metadata M_{VP} (line 31).

This step requires copying the VC_{tr} and the k entries related to the disclosed attributes into the VP metadata M_{VP} , signing the credential VC_{tr} and the metadata M_{VP} , and creating a new VP data structure VP_{tr} embedding the credential VC_{tr} , the VP metadata M_{VP} , and the signature P_{VP} . The size of the VP is proportional to the size of the data for disclosing k fields, i.e., k tuples $(claim_i, val_i, nonce_i, proof_i)$, plus the tree root, and the signature.

3) Verifier: Credential verification. The *verifyPresentation()* and the *verifyCredential()* functions (lines 4 and 1 of Algorithm 5, respectively) verify the signature of VP_{tr} and VC_{tr} , respectively. If the signature is successfully validated, the verifier checks that the values of the disclosed claims are correct and their hash belongs to the tree root (by using the *verifyTreeAttributes* function in line 10). In particular, the function retrieves the tree root from the Verifiable Credential VC_{tr} included in the presentation VP_{tr} (line 12), and the set of disclosed claim information from the presentation metadata M_{VP} (line 13), each consisting of a tuple $(claim_i, val_i, nonce_i, proof_i)$ which denotes the claim's clear

value, the nonce used to hash the claim value, and the proof to verify that the hashed claim belongs to the tree root. In the case of a Merkle tree (line 14), the verification of the presentation succeeds if the proof $proof_i$ proves that the leaf paired with $claim_i$, val_i , and $nonce_i$ belongs to the tree whose root is reported in VC_{tr} (function *MT.verifyProof*, line 16). Alternatively, in the case of a Merkle Patricia Trie (line 17), the proof is used to verify that $claim_i$ belongs to the tree root and to return the corresponding value val paired to $claim_i$ in the tree (line 18). The verification succeeds if the value val retrieved from the tree equals the hash value val_i and $nonce_i$ (lines 19-20). This step requires k inclusion proof verification and the verification of the VC_{tr} and VP_{tr} signatures.

E. SIGNATURE-BASED

For the signature-based approaches, we evaluated a method integrating BBS signature, which is among the most efficient signature schemes [71] available today and it is currently undergoing standardization efforts [72]. The BBS signature supports unlinkability and selective disclosure without the need to generate any additional parameter, except for a random nonce for the subject. Exploiting the BBS signature, a subject I can sign multiple messages whilst producing a single and constant size signature. Subsequently, the possessor of such a BBS signature (that could be another subject S) can create a zero-knowledge proofs-of-knowledge of the signature, while selectively disclosing subsets of the signed messages [40]. In this approach, we integrate BBS-signed data into VCs and VPs following the same operation flow as the other approaches in order to make a better quantitative comparison. The Verifiable Credential derived by this approach has a constant size as in the hash tree-based approach, and given its natural support to selective disclosure the issuer does not require to compute any extra parameters for each claim (such as keys and nonces in the previous approaches). However, while in the previous approaches the subject produces the Verifiable Presentation simply by copying a subset of the entries of the attributes data structure in the metadata of the presentation itself, in the BBS signature approach the subject has to compute a proof on their premise. The functions used in this approach are reported in Algorithm 6 while the VCs and VP production workflow is depicted in Figure 7.

1) Issuer: Preparing the credentials. In this approach, the issuer initially generates a BLS private-public key pair (sk_{BLS}, pk_{BLS}) to sign the claims with the BBS scheme. This key-pair will be used to sign claims for a given period of time. The issuer specifies as input of the *createCredential* function (line 1) in Algorithm 6 the claims C_{VC} , the metadata M_{VC} of the credential, the credential signing function *sign*, and the keypair (sk_{BLS}, pk_{BLS}) . Being C_{VC} the set of claims and M_{VC} the metadata of the Verifiable Credential, the issuer signs with the BBS scheme the set C_{VC} using the previously generated BLS key pair, producing the signature $signature_{BBS}$ (line 2). The signed claim set C_{VC}^{BBS} consists of the public key pk_{BLS} and the signature $signature_{BBS}$ (lines 3-4). The issuer signs (C_{VC}^{BBS}, M_{VC}) with the key used to sign credentials

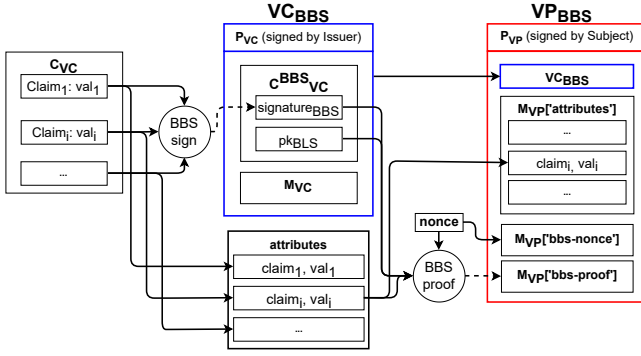


FIGURE 7. Illustration of the BBS-based selective disclosure. Plain arrows represent copy operations and dashed arrows results in a computation. A blue box represents a signed VC, a red box a signed VP.

and generates the proof P_{VC} (line 5) and then builds the Verifiable Credential $VC_{BBS} = \langle C_{VC}^{BBS}, M_{VC}, P_{VC} \rangle$ (line 6). The *attributes* set contains the list of claims with their plain values.

$$VC_{BBS} = \langle C_{VC}^{BBS}, M_{VC}, P_{VC} \rangle$$

$$attributes = \{(claim_1 : val_1), \dots, (claim_n : val_n)\} \quad (11)$$

Finally, VC_{BBS} and the *attributes* data structure are sent to the subject. As a result, issuing a new credential requires the computation of the BBS signature over the set of n claims and the signature of the credential.

2) Subject: Selective disclosure. To disclose to a verifier a subset of claims $disclosedAttr \subseteq attributes$, the subject creates a Verifiable Presentation through the *createPresentation* function (line 10). This function, generates a BBS proof with the *BBSproofGen* function (line 17) that requires *attributes* set, the indexes of the claims to disclose, the signature $signature_{BBS}$ and public key pk_{BLS} (which are retrieved in line 14 and 15 from VC_{BBS}), and a random nonce (created in line 16). The function stores the set *disclosedAttr* in $M_{VP}[attributes']$ (line 12), the BBS proof in $M_{VP}[bbs - proof']$ (line 19), and the nonce in $M_{VP}[bbs - nonce']$ (line 18) and then signs VC_{BBS} and M_{VP} (line 20) thus generating the proof P_{VP} required for building the Verifiable Presentation (line 21):

$$VP_{BBS} = \langle VC_{BBS}, M_{VP}, P_{VP} \rangle \quad (12)$$

In summary, this procedure requires the subject to generate a BBS proof, to copy the k entries related to the disclosed attributes, the random nonce, and the generated BBS proof into the VP metadata M_{VP} , to copy the credential VC_{BBS} , to sign the credential VC_{BBS} along with the metadata M_{VP} , and to create a new VP data structure VP_{BBS} embedding the credential VC_{BBS} , the VP metadata M_{VP} , and the signature P_{VP} . As a result, the size of the VP is proportional to the list of disclosed claims, plus the BBS proof, and the signature.

3) Verifier: Credential verification. The *verifyPresentation*() (line 24) and the *verifyCredential*() (line 30) functions verify the signature of VP_{BBS} and VC_{BBS} respectively. In the case of a VP, if the VP signature is successfully validated

Algorithm 6: Functions of the BBS-based approach

```

1 def createCredential( $C_{VC}$ : set,  $M_{VC}$ : set,  $sign$ : fun,
  ( $sk_{BLS}, pk_{BLS}$ ): keyPair):
2    $signature_{BBS} = BBSsign(C_{VC}, sk_{BLS})$ 
3    $C_{VC}^{BBS}[bbs - signature'] \leftarrow signature_{BBS}$ 
4    $C_{VC}^{BBS}[pk - bls'] \leftarrow pk_{BLS}$ 
5    $P_{VC} = createProof(C_{VC}, M_{VC}, sign)$ 
6    $VC_{BBS} = \langle C_{VC}^{BBS}, M_{VC}, P_{VC} \rangle$ 
7    $attributes = C_{VC}$ 
8   return  $\{VC_{BBS}, attributes\}$ 
9
10 def createPresentation( $VC_{BBS}$ :  $\langle C_{VC}^{BBS}, M_{VC}, P_{VC} \rangle$ ,  $M_{VP}$ :
  set,  $sign$ : fun,  $attributes$ : set,  $disclosedAttr$ : set):
11   for ( $claim_i : val_i$ )  $\in$   $disclosedAttr$  do
12      $M_{VP}[attributes'] \leftarrow (claim_i : val_i)$ 
13   end
14    $signature_{BBS} = C_{VC}^{BBS}[bbs - signature']$ 
15    $pk_{BLS} = C_{VC}^{BBS}[pk - bls']$ 
16    $nonce = randomKey()$ 
17    $proof_{BBS} = BBSproofGen(attributes,$ 
     $disclosedAttr.indexes, nonce, signature_{BBS}, pk_{BLS})$ 
18    $M_{VP}[bbs - nonce'] \leftarrow nonce$ 
19    $M_{VP}[bbs - proof'] \leftarrow proof_{BBS}$ 
20    $P_{VP} = createProof(VC_{BBS}, M_{VP}, sign)$ 
21    $VP_{BBS} = \langle VC_{BBS}, M_{VP}, P_{VP} \rangle$ 
22   return  $VP_{BBS}$ 
23
24 def verifyPresentation( $VP_{BBS}$ :  $\langle VC_{BBS}, M_{VP}, P_{VP} \rangle$ ,  $H$ :
  fun):
25   if  $verifyVP(VP_{BBS})$  then
26     return  $verifyBBSAttributes(VP_{BBS})$ 
27   end
28   return failure
29
30 def verifyCredential( $VC_{BBS}$ :  $\langle C_{VC}^{BBS}, M_{VC}, P_{VC} \rangle$ ):
31   return  $verifyVC(VC_{BBS})$ 
32
33 def verifyBBSAttributes( $VP_{BBS}$ :  $\langle VC_{BBS}, M_{VP}, P_{VP} \rangle$ ):
34    $pk_{BLS} = C_{VC}^{BBS}[pk - bls']$ 
35    $nonce = VP_{BBS}[bbs - nonce']$ 
36    $proof_{BBS} = VP_{BBS}[bbs - proof']$ 
37    $disclosedAttr = VP_{BBS}[attributes']$ 
38   if NOT  $BBSproofVerify(disclosedAttr, proof_{BBS},$ 
     $pk_{BLS}, nonce)$  then
39     return failure
40   end
41   return success

```

(line 25), then the verifier verifies the proofs on the disclosed claims by invoking the *verifyBBSAttributes* function (invocation in line 26, definition in line 33). In particular, such function retrieves from VC_{BBS} the BBS public key (line 34) and from M_{VP} the nonce, the BBS proof, and the disclosed claims (lines 35-37). Finally, it validates the proof with the *BBSproofVerify* function (line 38) that requires the parameters just mentioned. This step requires the verification of the signature of the credential and presentation, plus the execution of the *BBSproofVerify*.

VII. ETHEREUM JWT-BASED IMPLEMENTATION

An implementation of the selective disclosure approaches defined in Section VI has been developed in order to assess their performance and correctness. Indeed, the data model of the Verifiable Credential is designed to be independent of the proof format and it does not assume any particular digital proof or signature format. The proof mechanisms are based on external proofs or embedded proofs. The former encodes the Verifiable Credential data model onto another format, such as JWT, [73]. The latter format ensures the authenticity and integrity of linked data of the document by directly embedding a proof for them (such as the Linked Data Proofs mechanisms [74]). In the following, we focus on the external proof mechanism and, to increase the portability of data and interoperability among actors, the JWT standard is used to encode Verifiable Credentials and Presentations. Due to its wide usage in different domains, the Ethereum blockchain was selected as the best candidate to take the role of verifiable data registry and the implementation exploits a DID method based on Ethereum to identify the different actors.

A. DID METHOD BASED ON ETHEREUM

Ethereum supports smart contracts, programs whose execution is replicated by all the nodes of the blockchain. The `ethr` DID method, implemented by uPort [75], implements the verifiable data registry as a smart contract deployed on Ethereum⁴ (address: `0xdca7ef03e98e0dc2b855be647c3-9abe984fcf21b`). The DID method uses any Ethereum address or `secp256k1` public key as DID identifier while taking advantage of the immutability, pseudonymity, and cryptographic verifiability of data provided by the underlying blockchain. A DID for an Ethereum address is created by concatenating the string `"did:"` with the string representing the specific Ethereum did method (`"ethr:"`), the network identifier of the Ethereum blockchain (e.g., `"0x1:"` for the mainnet or `"0x5:"` for Goerli), and the Ethereum address as identifier. Since a DID is paired to a single Ethereum address, the identity behind such address maintains ultimate control over the DID. DIDs are resolved using the *ethr DID Resolver* [76], a program that takes in input a DID and returns the corresponding DID document. The basic structure of a DID document is computed on demand, while additional public keys and custom attributes are obtained from the registry smart contract. The creation of a DID is done on-premise and it does not involve the registry, therefore requiring no transactions and no blockchain fee to pay. Resolving a DID does not require blockchain fees as well since it only reads data from the registry. However, modifying the Ethereum DID Registry does have fees because it needs to modify the registry smart contract. Changing the ownership of a DID, delegation to another DID, and adding public keys are examples of actions involving writing operations onto the DID

⁴<https://github.com/uport-project/ethr-did-registry> [Accessed on 10 November 2025].

registry, and therefore requiring to pay a fee. Cryptographic keys used for DIDs authentication are generated locally by each entity, and securely stored in the user wallet. The public keys are not distributed directly but are instead retrieved from the corresponding DID Document.

In our implementation, the registry is deployed on a local Ganache network⁵ that simulates Ethereum. The implementation is done with NodeJs, and we used the native `crypto` library to import the cryptographic operations such as hashing and symmetric encryption. We also imported the following libraries: `merkletreejs`⁶ from the `merkletreejs` repository for Merkle trees; `merkle-patricia-tree`⁷ from the EthereumJS repository for Merkle Patricia Tries; `bbs-signatures`⁸ from Matrglobal repository that implement BBS+ signatures, a provable secure version of BBS [77] proposed by Au et al. [78].

B. IMPLEMENTING CREDENTIALS WITH JWT

JWT is a standard defined by Internet Engineering Task Force (IETF) to represent compact and portable claims in JSON between two parties [73] and it can be efficiently used as a model for Verifiable Credentials. The claims defined by a JWT can be secured with digital signatures using JSON Web Signature (JWS) or with encryption using JSON Web Encryption (JWE), depending on the security requirements of the claims. A JWT consists of a header, a payload, and a signature, each separated by dots (.). The header describes the type of token and the algorithm used to sign the JWT (e.g., ES256K, EdDSA, or ES256K-R), while the payload contains the claims. Finally, the signature of the issuer is obtained by encoding the first two components of the JWT (i.e., header and payload) with Base64Url and using them as input of the specified signing algorithm. We used the `did-jwt-vc` [79] library to represent in JWT format the Verifiable Credentials and presentations conforming to the W3C specification. In the proposed implementation, communications between entities are assumed to occur over secure channels and keys used in HMAC, BBS, and for symmetric encryption are securely handled by the credential holder. As a result, keys used for selective disclosure along with a VP are encrypted and never exposed in plaintext during transmission.

VIII. EVALUATION

This section experimentally evaluates and discusses the overhead introduced by the selective disclosure methods described in Section VI and implemented by exploiting the libraries presented in Section VII. The code we used to run our experiments, along with a detailed guide to use and extend

⁵Ganache: <https://trufflesuite.com/ganache/> [Accessed on 10 November 2025].

⁶<https://github.com/merkletreejs/merkletreejs> [Accessed on 10 November 2025]

⁷<https://www.npmjs.com/package/merkle-patricia-tree> [Accessed on 10 November 2025]

⁸<https://github.com/matrglobal/bbs-signatures> [Accessed on 10 November 2025]

it as well as to replicate our experiments, are available at: <https://github.com/andreadesalve/selectiveDisclosureSSI>.

A. METHODOLOGY

The metrics considered for the evaluation of the presented approaches are creation time (in ms), document size (in bytes), and verification time (in ms). We computed such metrics for both Verifiable Credential (VC) and Verifiable Presentation (VP) encoded as JWTs and signed with ES256K scheme. Each claim within the credential is associated with a value encoded as a fixed-length string of 10 characters. The decision to focus on short claim values in our experiments reflects realistic and practical use cases. Indeed, in many real-world scenarios, credential attributes such as name, age, country, gender, code, or role are typically represented by short values and require only a few characters. The creation time and the size of a VC are measured by executing the *createCredential()* function specified for each approach in Section VI. Similarly, the creation time and the size of a VP are measured by executing the *createPresentation()* function. The verification time is measured by using the *verifyCredential()* function for VCs and the *verifyPresentation()* function for VPs.

To better analyze the differences among the presented approaches, we conduct several experiments by varying the number of claims N embedded in a VC in the range $\{2^i : i \in [1, 10]\}$ and the percentage of claims that we disclose in the set $\{100\%, 75\%, 50\%, 25\%\}$ (in each experiment we randomly choose which are the claims of the VC that we disclose). Furthermore, we compared the overhead introduced by these approaches against a baseline that does not perform selective disclosure (labeled as *Baseline* in the figures). Since the presented approaches (but the atomic-based one) can be configured to provide different levels of security, we performed several experiments by varying the configuration parameters of each approach, as shown in Table 5.

In the case of the atomic-based approach for selective disclosure there are no parameters to be configured because a credential contains only a single claim, thus requiring a signature for each claim.

The hash-based approach exploits the HMAC function for hashing the claim values. In our experiments it has been configured to use either SHA3-256 with a 512 bits key, SHA3-512 with a 512 bits key, or SHA3-512 with a 1024 bits key. These configurations have been chosen for our experiments because they are proved to be secure and offer widely accepted security guarantees, including collision resistance, pre-image resistance, and second pre-image resistance. A detailed security analysis is shown in [80]. Furthermore, HMAC is considered secure with respect to unforgeability and key-recovery attacks when both it is instantiated with such proved hash function and the length of the HMAC key is at least 128 bits [66].

The encryption-based approach relies on AES to encrypt a claim, and in our tests we configured it for using key sizes of 128-bits, 192-bits, and 256-bits. These encryption schemes have been selected for our experiments since they are secure

and suitable for most practical uses, and they are currently used in many security critical scenarios. As a matter of fact, [81] states that the AES algorithm is suitable to protect classified information up to the "secret" level when using keys of length 128, 192 and 256 bits. Moreover, the same algorithm is considered suitable to protect "top secret" information when the key length is 192 or 256 bits.

The hash tree-based approach has been implemented by using either the Merkle Tree or the Merkle Patricia Trie approaches, which both take as input a couple of hash functions, H_l and H_t , used, respectively, on the leaves and on the internal nodes on the tree. The couples of algorithms (H_l , H_t) we used in our experiments for the Merkle Tree are: (keccak256, SHA3-256), (SHA3-256, keccak256), and (SHA3-256, SHA3-512). The Merkle Patricia Trie is configured to use two different implementations: the base version of the Merkle Patricia Trie (BaseTrie) and the enhanced version (SecureTrie) which stores claims values under the keccak256 hash of their keys for balancing tree. The couples of algorithms (H_l , H_t) we used in our experiments for the BaseTrie Merkle Patricia Trie are (keccak256, keccak256) and (SHA3-512, keccak256), while for the SecureTrie version we used (SHA3-512, keccak256). The hash and HMAC functions are provided by the crypto module of the Node.js library⁹, which defines a set of wrappers for OpenSSL.

The Keccak hash function is widely used in Ethereum [37] and it is closely related to SHA3, as it is based on the same construction and offers equivalent levels of cryptographic security.

Finally, the BBS-based approach can be implemented on any type 3 pairing-friendly curves (e.g., Elliptic curve), which are the most common approach in modern pairing-based cryptography. In order to guarantee high security level for users, our implementation exploits the BLS12-381 curve, which as a curve achieves around 117 bits of security [72], resulting in a private key of 32 bytes, a public key of 96 bytes, and a signature of 112 bytes.

Obviously, the previously mentioned configuration choices could affect the results of the experimental campaign. However, to mitigate this issue, we have conducted our experiments taking into account three distinct configurations for the hash-based, encryption-based, Merkle Tree-based, and Merkle Patricia Trie-based approaches. Moreover, it is also worth mentioning that our results are also inevitably dependent upon our implementation choices, e.g., the libraries adopted for hashing, encrypting and managing trees, the library adopted for credential representation, and so on.

Each measurement illustrated in the following sections is the average value of 30 runs. The experimental setup we adopted for our tests consists of IntelTM Core i7-4702MQ quad core CPU@2.20GHz equipped with Linux Ubuntu and having 16GB of RAM. Adopting a different experimental

⁹Crypto module of Node.js: <https://nodejs.org/api/crypto.html> [Accessed on 10 November 2025]

Approach	Method	Name	Nonce/ Key (bit)	Hashing	Encryption
Hash	Cryptographic hash function	Hash C_1	512	SHA3-256	-
		Hash C_2	512	SHA3-512	-
		Hash C_3	1024	SHA3-512	-
Encryption	Symmetric encryption	Enc C_1	128	-	AES
		Enc C_2	192	-	AES
		Enc C_3	256	-	AES
Hash Tree	Merkle Tree	MT C_1	256	$H_l = \text{keccak256} \mid H_r = \text{SHA3-256}$	-
		MT C_2	256	$H_l = \text{SHA3-256} \mid H_r = \text{keccak256}$	-
		MT C_3	256	$H_l = \text{SHA3-256} \mid H_r = \text{SHA3-512}$	-
	Merkle Patricia Trie	MPT C_1	256	BaseTrie: $H_l = \text{keccak256} \mid H_r = \text{keccak256}$	-
		MPT C_2	512	BaseTrie: $H_l = \text{SHA3-512} \mid H_r = \text{keccak256}$	-
		MPT C_3	512	SecureTrie: $H_l = \text{SHA3-512} \mid H_r = \text{keccak256}$	-
Signature	Short Group Signatures	BBS	768/256	-	-

TABLE 5. Experimental configurations used for the evaluation of the hash, encryption, and hash tree-based approaches.

setup could produce different results. For instance, executing our test on resource constrained devices could result in higher execution times for VC/VP creation and verification. For what concerns VC/VP sizes, instead, the results of the experiments represent a useful indication to understand whether the selective disclosure techniques studied in this paper can be adopted even on constrained devices. Summarizing, the discussions and results presented in the rest of the paper refer to the specific frameworks used for the implementation and the devices employed in the experiments. While results may vary when using different frameworks or hardware, they remain valid within the context of the chosen technological setup, providing useful and replicable insights for similar scenarios.

B. RESULTS ON VERIFIABLE CREDENTIALS

Figure 8 shows the average creation time, size, and verification time (y-axis) of a VC varying the number of claims embedded in the VC itself (x-axis) for all the configurations shown in Table 5. As shown in Figure 8a, compared to the baseline solution (black line in the figure), all the considered approaches introduce an overhead at the time of VC creation in order to support selective disclosure.

In particular, the BBS-based approach has the highest VC creation time because it requires signing multiple messages and aggregating them in order to produce a unique digital signature. Indeed, it exposes approximately the same level of security and complexity as the regular Elliptic Curve Digital Signature Algorithm (ECDSA) [82]. The creation of VC by using the atomic-based approach introduces also a high overhead with respect to the baseline solution because it requires the creation of a distinct VC, and hence a signature, for each claim.

Compared to the baseline solution, another approach that has a heavy impact on the VC creation time is the hash tree-based approach implemented by using Merkle Patricia Trie. Indeed, the three tested configurations (MPT C_1 , MPT C_2 , and MPT C_3) exhibit a very similar VC creation time, which is lower than the atomic-based approach, and higher than the hash-

based approach. Indeed, as explained in Section VI-D, all the configurations of the MPT require the execution of operations to: hash each individual claim's value by using HMAC; build the Trie by using claims' names as keys; sign the root of the Trie; and retrieve for each claim the proof of existence visiting the Trie.

The hash tree-based approach implemented using Merkle Tree turns out to be less expensive than the one implemented using Merkle Patricia Trie because claims' names are not used to index the tree, but they are stored directly on the leaves of the tree along with the claims' values. Furthermore, in the case of Merkle Trees, the slope of the curve increases rapidly as the number of claims in the VC increases.

The average creation time of a VC with the hash-based approach is higher than the time required by the encryption-based approach. This is due to both the use of HMAC construction in the hash-based approach and the computation of the signature. In particular, the HMAC requires two evaluations of the hash algorithm and the derivation of two keys from a secret key, which introduces an additional overhead (see Sec. VI-B). Moreover, when short claim values are used as in our experiments, the signature operation takes more time in the case of hash-based approach because the resulting hashed data to be signed is larger than the encrypted data in the encryption-based approach. This is due to the fixed-length output of cryptographic hash functions, which increases the size of the original claim values (fixed to 10 characters in our experiments). As the size of the claim values increases, the encryption-based approach is expected to require more time to generate a VC than the hash-based approach. This is due to the larger size of the input passed to the signature algorithm by the encryption-based approach, which increases the cost of the signature phase. Experimental results not included in this paper confirm this expectation by revealing that for very large claim values (equal or greater than 3000 characters), the hash-based approach outperforms the encryption-based one in terms of VC creation time.

The average creation time of VC by using either the Encryption-based or hash tree-based approach (implemented

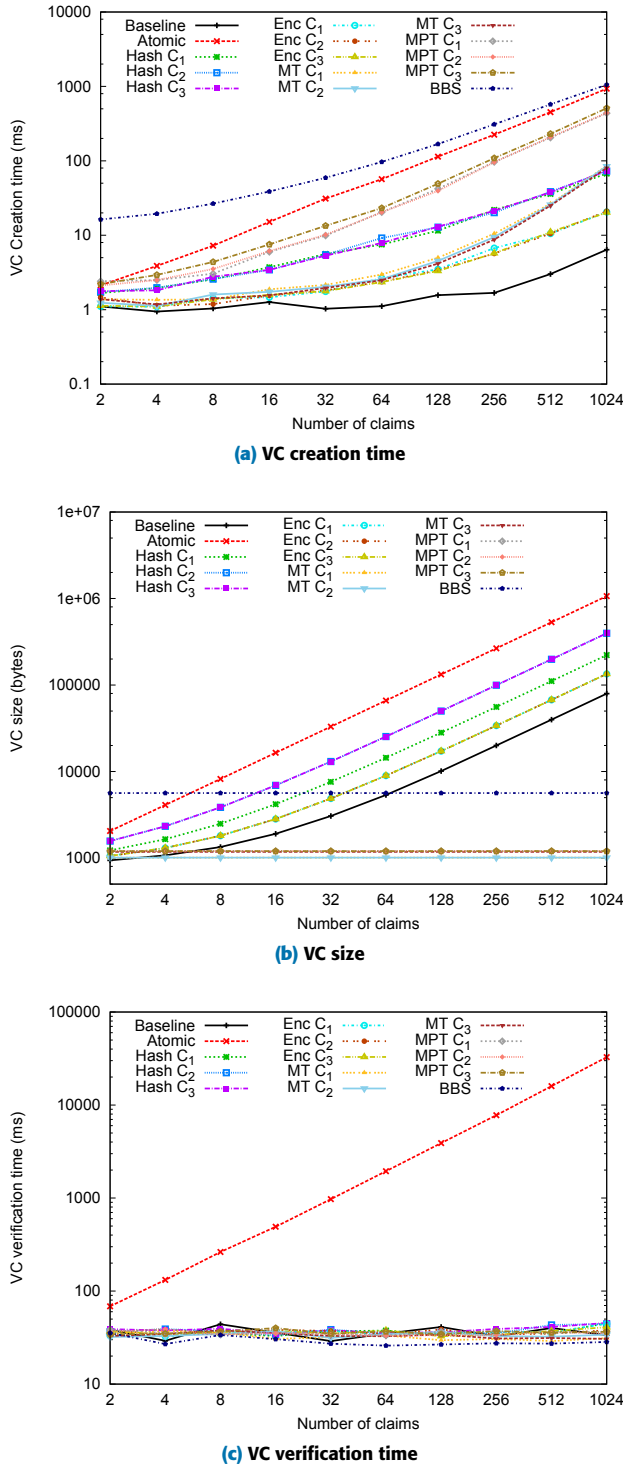


FIGURE 8. Average time (ms) to generate and verify a VC, and size (bytes) of a VC varying the number of embedded claims.

with MT) is very similar when the number of claims in a VC is below 64 claims, resulting in an average creation time between 1 ms and 5 ms. Instead, for credentials with more than 64 claims, the creation time of the hash tree-based approach implemented by using MT is increasingly larger than the one of Encryption-based approaches.

Figure 8b reveals that hash tree-based approach reduces the final size of the VC with respect to the baseline solution because the VC consists only of the root of the tree. Indeed, the root of the tree is obtained by recursively aggregating and hashing all claims of the VC and, compared to the baseline solution, the gain obtained from this aggregation increases as the number of claims in the VC. As a matter of fact, the final VCs produced exploiting the hash tree-based approach have a constant size of about 1200 bytes (notice that the plots of the hash tree-based approaches overlap in the figure) that does not depend on the number of claims in a VC. The same holds for the BBS-based approach, whose VCs always require more space than the hash tree-based approaches (i.e. about 5644 bytes).

For the other approaches, the VC size increases with the number of claims. First of all, we observe that the fixed-size VCs produced with the hash tree-based approach take less space than the ones produced with all the other approaches when the number of claims is 8 or more, while the VCs produced by the BBS approach always take more space than the ones produced with the hash tree-based approaches, but they require less space than all the other approaches when the number of claims is equal to or greater than 64. The atomic-based approach introduces the highest overhead in VC size for 8 or more claims because each claim requires a distinct signature.

Figure 8b shows that also the hash-based approach has a significant impact on the size of the VC because each VC stores all the hashed claims' values, which in our case appear to be larger than the clear values of the claims (i.e., the baseline approach). As a matter of fact, for 32 claims or more, the size of a VC produced with the hash-based approach is greater than the size of the VCs produced using the other approaches, except for the atomic one. In particular, the space required by the hash-based approach for representing a VC depends on the output size of the hashed claim. For example, both the configurations Hash C₂ and Hash C₃ use SHA3-512, resulting in an output fixed-length hash of 512 bits for each claim. For this reason, increasing the size of the claim values in the hash-based approach does not affect the overall size of the VC.

The size of the VCs obtained using the encryption-based approach are overlapped in Figure 8b and it is smaller than the VCs size of the hash-based approach. In particular, the encryption-based approach increases the size of the final VC by a constant factor with respect to the baseline approach, regardless of the key size used to encrypt claims' values. Indeed, it creates for each claim a ciphertext that has at most the same size as the claim plus a block size of 16 bytes for padding. Specifically, another experiment we conducted (not shown in this paper) reveals that the encryption-based and hash-based approaches generate credentials of the same size when the length of the claim values is set to 60 characters. Note that the figure does not take into account the size of the *attributes* data structure that includes the claims in clear text and the keys to verify them, because such data structure is

not embedded in the VCs. However, the size of the *attributes* data structure has an impact on VPs size, as shown in the following.

Figure 8c shows the time required by all the approaches to verify a VC. The verification of a VC involves only the verification of the digital signature, and we observe that for all the approaches but the atomic-based one the verification time does not depend much on the number of claims in a VC. The approach that requires more time to verify a VC with respect to the baseline solution is the atomic-based approach, where the verification time increases as the number of claims. Indeed, each claim of the VC is signed and verified separately, resulting in a verification time linear with respect to the number of claims.

C. RESULTS ON VERIFIABLE PRESENTATIONS: REVEALING 100% OF CLAIMS

The subject creates a VP in order to share with a verifier one VC (or more) and the information necessary to selectively reveal a subset of the VC's claims only. Given a VC consisting of N claims (x-axis), Figure 9 shows the costs introduced by the selective disclosure approaches in order to reveal **all** the claims of a VC, representing the worst case scenario resource-wise. Section VIII-E presents the same set of measurements varying the percentage of claims to disclose.

The time required to create a VP and the VP size are shown in Figures 9a and 9b, respectively. In contrast to the baseline solution, selective disclosure approaches reveal the value of protected claims in the VC by adding to the VP extra information for disclosing only the selected subset of claims of the VC, increasing both the VP creation time and the final size of the VP. As concerns the VP creation time, Figure 9a reveals that the approaches used for selective disclosure (but the BBS-based one) and the baseline solution exhibit similar execution time when the VC consists of 8 claims or less and VP creation time ranges between 1 ms and 5 ms. The differences between the VP creation times of the proposed approaches are more relevant for VC consisting of 16 claims or more.

The BBS-based approach turns out to be the most computationally expensive approach for creating a VP (taking 4560 ms to create a VP related to 1024 claims), as it requires the subject to compute a zero-knowledge proof based on the VC's signature which allows to selectively disclose a subset of the signed claims in a VC. As shown in Figure 9b, the size of the VP produced with the BBS-based approach turns to be among the smallest ones as the number of claims increases (e.g., it is about 550 KB for a VP revealing 1024 claims) because the proof size increases by a constant factor proportional to the length of the private key, each time a claim is revealed. The hash tree-based approach (all configurations) is the second most expensive approach in terms of time required for creating a VP and, for 16 claims or more, is the most expensive approach in terms of VP size. Indeed, it creates a VP that consists of a set of proofs, one proof for each claim to be revealed in the VP demonstrating the claim belongs

to the root tree. Since the size of the proof depends on the height of the tree, this operation has a significant impact on the final size of the VP and takes between 300 ms and 600 ms for VC consisting of 1024 claims, depending on the specific configuration. Indeed, the final size of the VP (see Figure 9b) results to be very large (between 8 MB and 4 MB for VC consisting of 1024 claims) and it mainly depends on both the number of claims to be revealed in the VP and the length of the output produced by hash functions.

The time and the size taken to create a VP by the atomic-based, hash-based, and encryption-based approaches are comparable among them and it ranges between 19 ms and 120 ms if we consider VP that discloses 1024 claims. In particular, the overhead introduced by the atomic-based approach for the creation of a VP includes the time for the selection of the claims to be disclosed and the time for creating a VP for the disclosed claims, including the time for the signature of the VP itself. For this reason, the final size of the resulting VP of the atomic-based approach (overlapped with Hash C_3 when the number of claims is greater than 8) may be quite large (about 1.5 MB for VP revealing 1024 claims) and it depends on the number of claims disclosed by the subject.

The VP created by using the hash-based and encryption-based approaches consists of a VC embedding all the hashed/encrypted claims, along with the information required to reveal all of them to the verifier. As shown in Figures 9a and 9b, both the hash-based (except Hash C_3) and the encryption-based approaches require shorter creation time and produce smaller presentations than the atomic and hash tree-based approaches. The VP creation time exploiting the Hash C_3 is slightly greater than the one of the Atomic approach for 256 claims or less, while the Hash C_3 credential size is greater than the Atomic credentials size for 8 claims or less, and it is the same for 16 claims or more. Both the hash-based and encryption-based approaches encode the information required to reveal a subset of the claims as a set of triples in the metadata of the VP, one for each claim to be revealed. In the case of the hash-based approach, a triple indicates the path to reach the claim, the clear claim value, and the nonce used to hash the claim value, while in the encryption-based approach a triple indicates the path, the symmetric key and the nonce used to decrypt the claim value. As a result, the plots confirm that VP creation time and the VP size in the case of hash-based and encryption-based approaches mainly depend on both the number of claims to be revealed and the cryptographic algorithm used to hash or encrypt the claims.

Figure 9c shows how long it takes for a verifier to verify a VP revealing all the claims in the VC (x-axis). The BBS-based approach takes the longest time to verify VPs (between 70 ms and 1200 ms) as the verifier requires checking that the provided proof is valid against a public BBS key for a set of disclosed VC's claims.

The VP verification times of the other approaches differ significantly from that of the BBS-based approach and they are very similar for VP consisting of less than 64 claims, ranging at most between 30 ms and 60 ms.

For VP consisting of more than 64 claims, the hash tree-based approach exposes a higher VP verification time compared to the other approaches and the implementation-based Merkle Patricia Trie exposes higher verification times than the implementation based on Merkle Tree. This is mainly due to the different types of data structures used by the approaches: Merkle tree exploits binary tree while Merkle Patricia Trie exploits prefix tree. In particular, the VP verification time related to a VC consisting of 1024 claims is at most 479 ms for the Merkle Patricia Trie and at most 275 ms for the Merkle Tree.

The VP verification time of the atomic-based, hash-based, and encryption-based approaches increases quite slowly with respect to the other approaches. In particular, in the case of the atomic-based approach, the verifier has to check the signature on all the VCs shared in the VP, taking at most 122 ms to verify a VP revealing 1024 claims. In contrast, the hash-based and the encryption-based approaches take at most half the time (60 ms) to verify the same VP. Indeed, the hash-based approach requires to re-compute the hash of the revealed claims values (in $M_{VP}['attributes']$) and comparing the result with the corresponding hash values in VC_h , while the encryption-based approach needs to decrypt each claim's value with the provided key.

D. REDUCING THE VP SIZE IN THE HASH TREE-BASED APPROACH

Previous analyses revealed that some approaches for selective disclosure suffer from storage inefficiency, which is due to the large size of the final VP (see Figure 9b). In particular, this is the case of the hash tree-based approach, where the VP size is very high because it consists of a set of proofs to verify that the revealed claims belong to the tree. Each proof is related to a specific claim i to be revealed, and it consists of the claim's name ($claim_i$), the claim's value (val_i), the path from the leaf representing the claim to the root of the tree ($proof_i$), and the nonce value ($nonce_i$). Proofs are strings of data encoding paths on tree nodes, and they are suitable to be transformed using a succinct representation. To mitigate such a problem, this section further improves the hash tree-based selective disclosure approach by evaluating the advantage of using compression algorithms to reduce the VP size. Indeed, we defined a new version of the hash tree-based approach that handles the proof information needed to reveal the claims' values in a different way. Instead of directly storing this information in plain text, the improved version of the hash tree-based approach compresses the proof using Brotli compressor [83] before being stored in the VP and signed by the subject of the VC. In order to evaluate the impact of this transformation on the proof of the VP, we implemented the improved version of the hash tree-based approach and collected the creation time, verification time, and the size required by users to reveal VCs consisting of different number of claims (from 2^1 to 2^{10} claims). Figure 10 illustrates box plots to compare the performance results obtained by using different configurations of the Merkle Tree

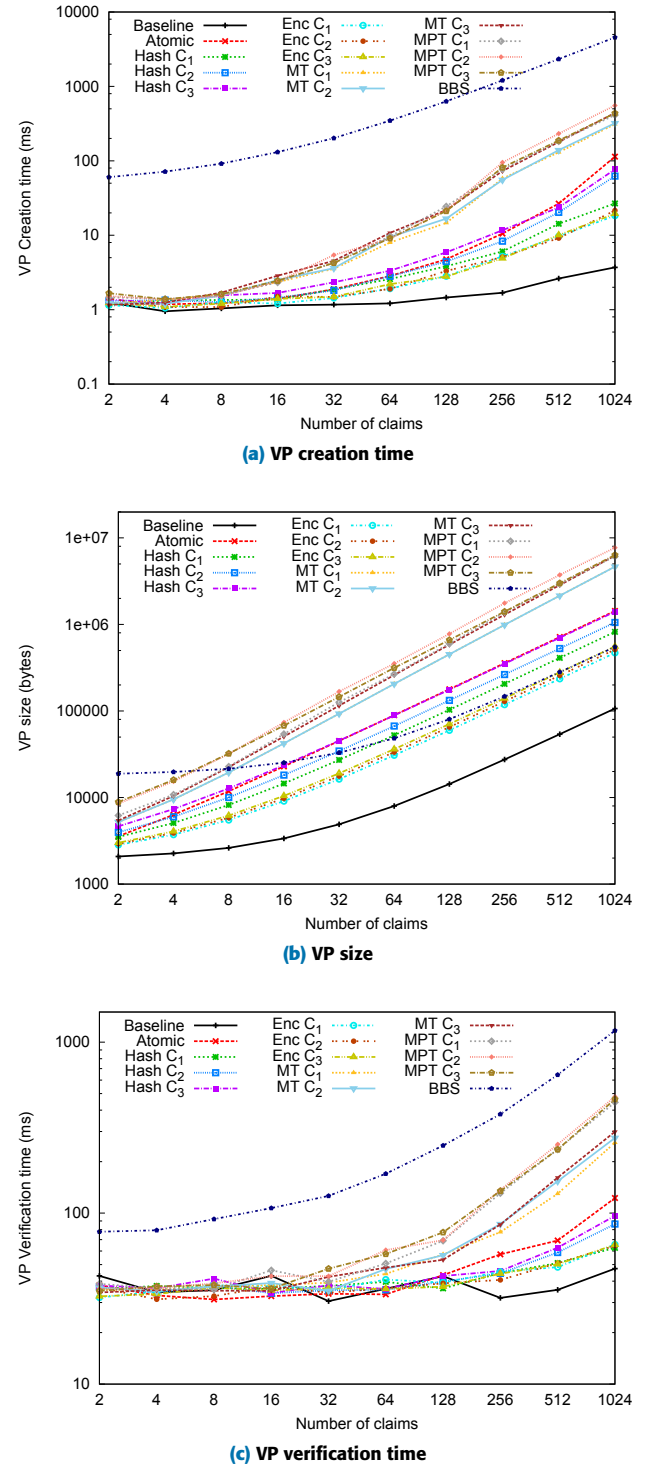


FIGURE 9. Average time (ms) to generate and verify a VP, and size (bytes) of a VP disclosing all the claims.

(MT) and Merkle Patricia Trie (MPT) implementations with compression (MT^C , MPT^C) and without compression. The impact of the compression on the VP creation time is shown in Figures 10a and 10d. The VP creation time of the MT and MPT without compression is less than 553 ms while in the case of compression the VP creation time increases of

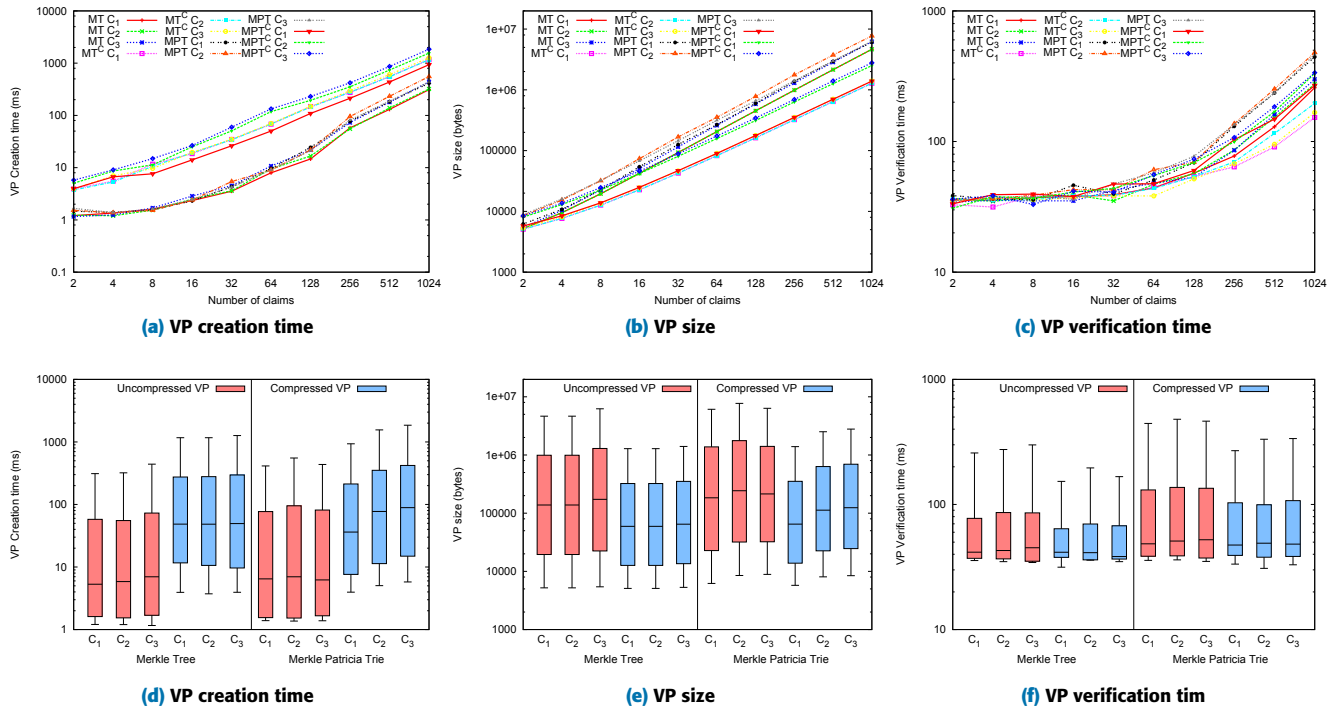


FIGURE 10. Impact of the compression on the size and the time needed to create and verify a VP using the hash tree-based approach for revealing all the claims of the VC.

at most 2000 ms.

However, as shown in Figures 10b, 10e, 10c, and 10f, the compression of the VP proof has the benefit of reducing the size of the final VP while keeping the verification cost similar to the uncompressed version of the approach. In particular, in the case of the MT, the reduction in the VP size is about the same among the different configurations and the size of the VP with compressed proof decreases by 50% starting from the VC with more than 32 claims. The MPT implementation exhibits different behaviors depending on the specific configuration, and the highest gain obtained by compression is provided by $MPT^C C_1$, which decreases the VPs size by 50% starting from VC consisting of more than 16 claims.

As shown in Figures 10c and 10f, the VP verification time of the VP does not increase with compression. This is due to the fact that verifying the signature of a VP is more expensive than verifying the proof of the revealed claims on the tree. As a matter of fact, more than 60% of VP verification time is spent on signature verification by MT and MPT algorithms without compression, regardless of the number of claims. The MT and MPT algorithms with compression create small VPs that take less time to be verified: more than 40% of VP verification time is spent on signature verification.

E. RESULTS ON VERIFIABLE PRESENTATIONS: REVEALING 25%, 50%, 75% OF CLAIMS

The previous set of experiments were focused on the worst-case scenario where the subject of a VC reveals to the verifier all the claims of the VC. In this section, we extend the

previous set of experiments by analyzing the performance of the approaches varying the number of claims the subject discloses to the verifier. In particular, we conducted three sets of experiments where we disclosed 25%, 50%, and 75% of the claims in the VCs. To improve the readability of the plots, for each approach we show only the configuration achieving the lowest VP creation time. Moreover, since VC creation time, VC verification time, and VC size do not depend on the number of claims that are disclosed, we refer to experiment shown in Section VIII-C, and in this section and we show only the VP size, the VP creation time, and the VP verification time. Figure 11 shows the performance achieved by the chosen configurations for each disclosure ratio, including the case where we disclose all the claims in the VC (100%).

The VP creation times of the encryption-based, atomic-based, and hash-based approaches are the lowest and comparable when the number of claims in the VC is lower than 2^8 or the fraction of revealed claims is equal to or less than 50%. The atomic-based approach exposes higher VP creation time than the hash-based and encryption-based approaches when the number of claims in the VC is higher than 2^8 and the fraction of revealed claims is greater than 50%. The compressed versions of the hash tree-based approach ($MT^C C_1$ and $MPT^C C_1$) described in Section VIII-D exhibit similar performances for what concern the VP creation time and they are slower than the encryption-based, atomic-based, and hash-based approaches, regardless of the fraction of revealed claims. Finally, the signature-based approach results to be the most expensive in terms of VP creation time.

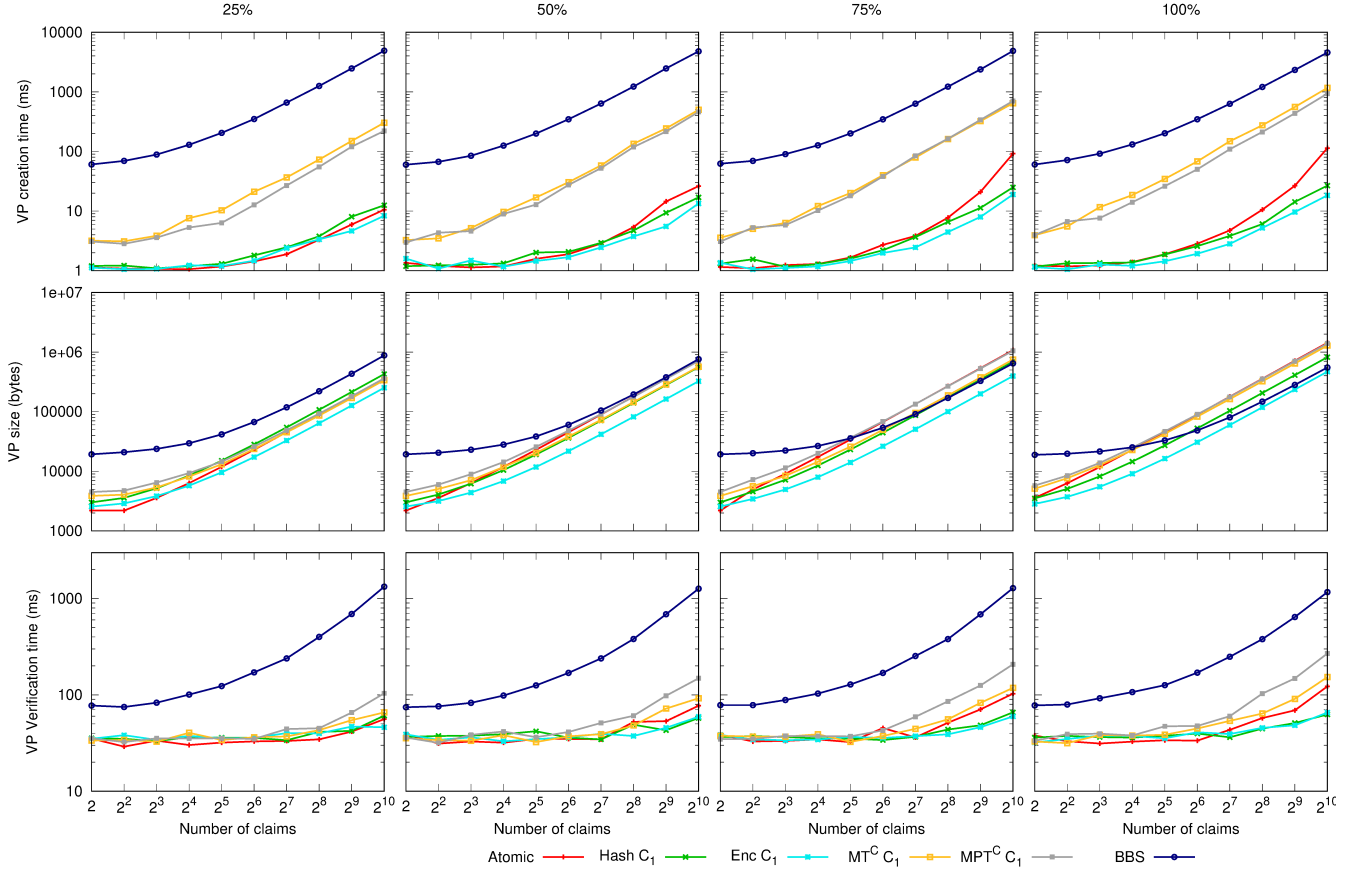


FIGURE 11. VP creation and verification time and VP size required to present 25%, 50%, 75%, and 100% of the claims of a VC, varying the number of such claims in the range $\{2^i : i \in [1, 10]\}$.

The encryption-based approach exposes almost the lowest VP size, regardless of the total number of claims and the fraction of revealed ones. When the fraction of the revealed claims is equal to or lower than 50%, the sizes of the VP obtained by using the hash-based, the hash tree-based, the atomic-based are comparable and the signature-based approach results to be the most expensive in terms of VP size. In contract, as the number of VC claims is higher than 2^5 and the fraction of revealed claims is above 50%, the size of the VP of the BBS-based approach is not the largest one. In particular, for VC consisting of more than 2^5 claims and when the fraction of disclosed claims is 75%, the VP created by using the atomic-based, the Merkel Tree and the hash-based approaches results in having a similar size than the ones adopting the BBS-based approach, while the VP created with the Patricia Trie approach are slightly bigger. Instead, when the fraction of disclosed claims is 100%, for VC embedding more than 2^5 claims, the size of the VP produced with BBS-based approach is greater of the size of the VP produced with the encryption-based approach only, while it smaller than all the others.

The VP verification time required by the encryption-based approach is comparable to that of atomic-based, hash-based, and encryption-based, and hash tree-based approaches when the number of claims in the VC is lower than 2^6 , regardless

of the fraction of revealed claims. However, when the number of claims is equal or higher than 2^6 , the difference in time required by each approach to verify a VP becomes more noticeable. Indeed, as shown in Figure 11, the encryption and hash-based approaches expose similar results and they take the lowest VP verification time, while the VP verification time of the atomic-based approach is slightly greater. The VP verification time of the hash tree-based approach (again, for more than 2^6 claims) is higher compared to the atomic, hash, and encryption-based approaches, especially when the implementation is based on Merkle Patricia Trie ($MPT^C C_1$). Indeed, the $MPT^C C_1$ implementation requires more time to verify a VP than the $MT^C C_1$ and the differences between the two implementations increase as the number of the VC's claims increases. Finally, the signature-based approach results to be the most expensive in terms of VP verification time, regardless of the fraction of revealed claims.

IX. DISCUSSION

To better highlight our findings, we evaluate the impact on the actors of the system of the proposed approaches by discussing their usability and comparing their performance. The usability concerns the effort required by the actors of the system to adopt each approach, and it is measured by considering

the number of additional data required to protect and reveal the claims of a VC to the desired audience. The performance comparison concerns the computational time required for creation/verification of VCs and VPs (see Section VIII).

A. ISSUER

In typical real-world applications, the issuers of VC is a well recognized organization or service provider with extensive computational and memory resources. For these reasons, issuers are less concerned with the computational and storage resources required to integrate selective disclosure approaches. In addition, the VC Creation performed by issuers typically occurs less frequently than other operations since a VC, once created, is used multiple times until its expiration date.

The usability of the Atomic-based approach is high for the issuer because they only need to produce single-claim VCs in the same way as they produce multi-claim ones. However, the issuer must properly bind the credentials concerning the same set of claims to prevent the subject from perpetrating the bundle unrelated credential attack by wrapping together claims belonging to distinct sets, creating a false Verifiable Presentation for a verifier. Moreover, the overhead in terms of storage is high for the issuer, because this approach requires the creation of n VCs (each embedding the related signature) where n is the number of claims, which results in high memory consumption in case of a large set of claims. The usability of the hash-based and encryption-based approaches is medium, because the issuer has to configure a number of parameters that heavily impact the security level of the final solution, including nonces, keys length, encryption, and hashing primitives. Moreover, hash-based and encryption-based approaches increase the size of the original VCs because they increase the length of the claims' values substituting them with n hash values or encrypted values. Finally, such approaches also require the issuer to create an additional data structure (called *attributes*, see Sections VI-B and VI-C) whose size is proportional to the number of claims in the credential and, obviously, requires additional storage space. The approach based on hash trees has low usability because it requires the creation and storage of an additional tree data structure. The creation of such a tree data structure requires managing a high number of parameters (i.e., a nonce and a proof for each claim, where a proof consists of a number of digests equal to the height of the tree), that heavily impact the security level of the final solution. However, such an approach reduces the size of VCs since they do not include all the claims' values but only the tree root hash value. Indeed, in this approach as well, additional data (named *attributes* data structure in Section VI-D) required to perform selective disclosure are sent to the issuer separately. In the case of the BBS-based approach, the issuer is responsible for generating a key pair based on BLS curve. This approach reduces the size of VCs and increases their usability as well because VCs include only the BBS signature and BLS public key. Moreover, the additional data structure that is created for the

BBS-based approach only includes the plain text values of the claims.

For what concerns the performance comparison, we take into account the computational time required to create a new VC. The encryption-based approach exposes the lowest VC creation time. The computational time consumed by the tree and the hash-based approaches is medium because it involves the computation of several hash values. The atomic and the BBS-based approaches are the most expensive approaches for the issuer.

Finally, we investigated whether selective disclosure approaches can influence revocation of VC. Specifically, we found that the atomic-based approach has a notable impact on revocation mechanisms. Indeed, this approach requires a greater number of revocation operations (one revocation operation for each claim) compared to other approaches and such overhead may affect system scalability and performance, particularly in high-frequency credential usage scenarios. Unlike other methods, encryption-based, hash-based, hash tree-based, and BBS-based approaches avoid additional overhead for credential revocation, since all relevant data is embedded directly within an individual verifiable credential.

B. SUBJECT

We expect that the subject of a VC, who is also responsible for creating VPs, engages in VP creation very frequently, at least as often as VCs and VPs are verified. Each time the subject needs to prove a claim to a verifier, a new VP must be generated by the subject. In addition, it is very likely that subjects will interact with verifiers through the use of mobile devices with limited storage resources, such as smartphones or tablets. For this reason, the subject is affected by both storage size and processing time. The usability of the atomic-based approach is very high for the subject because they simply have to choose the set of claims (i.e., the VCs) that they want to embed in the VP. Hence, this approach only requires the subject to store one distinct VC for each claim. Indeed, this approach does not involve the management of any additional attribute for the subject. The usability of the hash, encryption, and hash tree-based approach is medium as the subject is burdened with the management of the additional data structure called *attributes*, which embeds the data to be copied in the VP to prove the real value of each disclosed claim. For the aforementioned approaches, the overhead incurred by the subject in terms of storage is equivalent to that of the issuer, as the VCs and additional data structures produced by the issuer to allow selective disclosure are transferred to the subject. The BBS-based approach has a low level of usability because, besides requiring the subject to store the data structure embedding the set of clear claims' values, to disclose a subset of claims it also requires the subject to create a BBS proof to be embedded in the VP.

For what concerns the performance in terms of computational time, we take into account the time required to create a VP. The atomic, hash, and encryption-based approaches have a low VP creation time. The computational time of the

hash tree-based approach is medium, while the BBS-based approach exhibits the highest VP creation time, because of the BBS proof creation.

C. VERIFIER

The verifier is involved in VC/VP verification every time the subject presents a VP. For this reason, it occurs typically very frequently, resulting in high computational and storage demands for the verifier. For this reason, the verifier is affected by both the storage size and processing time of the VP/VC verification. The Atomic-based approach provides a high usability level for the verifier, because the latter must only check the signatures of VCs and VP, and that the subject has properly bundled related claims, i.e., claims issued by the same issuer to the same subject but related to distinct claim sets (e.g., a presentation consisting of one claim related to a master degree and the other related to a professional certification from the same authority). However, the atomic-based approach has high storage overhead for the verifier as well, since the verifier receives one VC for each disclosed claim. The usability of the hash-based, encryption-based, and BBS-based approaches is medium. As a matter of fact, besides checking the signature of VCs and VP, for each disclosed claim the verifier must control that the value declared by the subject is actually the real one. To this aim, in the hash-based approach, the verifier computes the hash of the provided claim value and key, and the resulting digest is compared with the digest in the VC, while in the encryption-based approach, the encrypted claim value in the VC is decrypted with the provided key. In the case of the BBS-based approach, the BBS-proof provided by the subject is verified through a BBS primitive. The usability of the hash tree-based approaches is low, because the procedure to control that the provided claim value is the real one is more complex than the ones of the previous approaches.

From the computational time point of view, we take into account the time required to verify a VP. The BBS-based approach takes considerably more time for verifying a VP than the other approaches, due to the proof verification procedure. The VP verification time for the hash tree-based approaches is smaller than the one of the BBS-based approach, while the hash, encryption, and atomic-based approaches expose similar VP verification times, and they are faster than all the other approaches.

Finally, the analyzed selective disclosure approaches do not interfere with the existing VC verification mechanisms, whose behavior remains unchanged. In case of cross-domain credential verification, it may happen that credentials from distinct domains have been protected adopting different selective disclosure techniques. However, this is not an issue as long as the verifiers support the selective disclosure techniques adopted in the VCs they need to verify, since the verification of a VC is performed independently by a verifier within each domain and the investigated selective disclosure approaches do not rely on external services to protect the confidentiality of the claims. Specifically, such interoperability

across different domains is ensured by the fact that selective disclosure methods adhere to established standards, allowing any verifier to validate a VC using common frameworks such as the W3C VC Data Model [5] and DID^s [33].

X. CONCLUSION

This paper investigated the problem of selective disclosure of the claims embedded in a VC, analyzing and comparing five different approaches currently available in the scientific literature, which are based on: atomic credentials, hashing, encryption, hash trees, and BBS signature. In order to conduct a fair experimental comparison among such approaches, for each of them, we developed a reference implementation of the main SSI operations, and evaluated their performance under the same conditions. In particular, we measured the time taken to generate and to verify Verifiable Credentials and Presentations, and we evaluated the space required for their storage, varying the specific configuration parameter of each selective disclosure approach (e.g., the chosen cryptographic hash function for the hash-based, the Merkle Tree-based, and the Merkle Patricia Trie-based approaches), the number of claims embedded in each VC in the range $\{2^i : i \in [1, 10]\}$, and the percentage of claims disclosed by the subject in the set $\{25\%, 50\%, 75\%, 100\%\}$. Although the results we obtained in our experimental campaign are dependent on the specific hardware and software frameworks and configurations we adopted, we think that they still reveal interesting characteristics and behaviors of the selective disclosure approaches currently available in the literature, and can be of use to application designers for choosing which approach is more appropriate for their application scenarios. Overall, we observe that the adoption of VCs and VPs enabling selective disclosure always introduces a non negligible overhead in their creation time with respect to plain text ones, and the BBS-based approach is the one introducing the largest overhead. The Hash tree-based and BBS-based approaches create VCs of fixed size, independently on the number of claims they embed, while adopting the other approaches, the size of VCs is proportional to the number of embedded claims. The size of VPs is always proportional to the number of claims to be disclosed because, besides the VCs, the VPs also include the additional data required to implement the selective disclosure.

The analysis and evaluation of the selective disclosure approaches we conducted in this paper allowed us to identify some future research avenues for further advancement in this field. First of all, the selective disclosure approaches proposed in the current literature lack of uniform guidelines and specifications, which would ease the integration into real-world scenarios. A standardization of the selective disclosure approaches would ensure consistency and promote compatibility of the approaches among different SSI systems. In addition, such a standardization should provide to application designers the practical guidelines to integrate the approach into their credentials wallet, as well as the security recommendations about the key generation, nonce, cryptographic algorithms, and hash functions to be used.

Another research direction which would deserve further investigation involves the integration of negotiation mechanisms for selective disclosure. In particular, such mechanisms would help the holder of Verifiable Credentials to trade with a verifier a convenient set of credentials/claims that, on the one hand, is sufficient to access the requested service while, on the other hand, satisfies the holder's privacy needs, also taking into account the credentials/claims disclosed in previous interactions. Finally, selective disclosure approaches are designed to enable the holder of Verifiable Credentials to decide which claims to hide from the verifier. A further research direction concerning selective disclosure would involve the definition of advanced techniques allowing also the issuer of Verifiable Credentials to define their constraints concerning the verifiers that can read the information stored in the credentials they issue. In fact, a Verifiable Credential, in addition to the holder's personal information, could also reveal some information about the issuer of the credential, such as the nature of their relationships with the holder, the specific role the holder plays in relation to the issuer, and other relevant context information.

ACKNOWLEDGMENT

This work was partially supported by project SERICS (PE00000014) under the MUR National Recovery and Resilience Plan funded by the European Union - NextGenerationEU.

REFERENCES

- [1] D. Hardt. The OAuth 2.0 Authorization Framework. Online <https://datatracker.ietf.org/doc/html/rfc6749> [Accessed on 10 November 2025].
- [2] Alexander Mühle, Andreas Grüner, Tatiana Gayvoronskaya, and Christoph Meinel. A survey on essential components of a self-sovereign identity. *Comput. Sci. Rev.*, 30:80–86, 2018.
- [3] Blaž Podgorelec, Lukas Alber, and Thomas Zefferer. What is a (digital) identity wallet? a systematic literature review. In *2022 IEEE 46th Annual Computers, Software, and Applications Conference (COMPSAC)*, pages 809–818. IEEE, 2022.
- [4] Georgios Karopoulos, José Luis Hernández Ramos, Vasileios Kouliaridis, and Georgios Kambourakis. A survey on digital certificates approaches for the COVID-19 pandemic. *IEEE Access*, 9:138003–138025, 2021.
- [5] Manu Sporny, Dave Longley, and David Chadwick. Verifiable credentials data model v1.1 (W3C proposed recommendation 03 august 2021). Online <https://www.w3.org/TR/vc-data-model/> [Accessed on 10 November 2025].
- [6] Changlai Du, Hexuan Yu, Yang Xiao, Y Thomas Hou, Angelos D Keromytis, and Wenjing Lou. {UCBlocker}: Unwanted call blocking using anonymous authentication. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 445–462, 2023.
- [7] Jan Camenisch and Anna Lysyanskaya. Signature schemes and anonymous credentials from bilinear maps. In *Annual international cryptology conference*, pages 56–72. Springer, 2004.
- [8] Shafi Goldwasser, Silvio Micali, and Chales Rackoff. The knowledge complexity of interactive proof-systems. In *Providing sound foundations for cryptography: On the work of shafi goldwasser and silvio micali*, pages 203–225. 2019.
- [9] Andrea De Salve, Andrea Lisi, Paolo Mori, and Laura Ricci. Selective disclosure in self-sovereign identity based on hashed values. In *2022 IEEE Symposium on Computers and Communications (ISCC)*, pages 1–8, 2022.
- [10] Md Sadek Ferdous, Farida Chowdhury, and Madini O Allassafi. In search of self-sovereign identity leveraging blockchain technology. *IEEE access*, 7:103059–103079, 2019.
- [11] Kheng Leong Tan, Chi-Hung Chi, and Kwok-Yan Lam. Survey on digital sovereignty and identity: from digitization to digitalization. *ACM Computing Surveys*, 56(3):1–36, 2023.
- [12] Noot A Alissa and Waleed A Alrodhan. Self-sovereign identity management: A comparative study and technical enhancements. *International Journal of Computer Science & Network Security*, 23(12):27–80, 2023.
- [13] Jana Glöckler, Johannes Sedlmeir, Muriel Frank, and Gilbert Fridgen. A systematic review of identity and access management requirements in enterprises and potential contributions of self-sovereign identity. *Business & Information Systems Engineering*, 66(4):421–440, 2024.
- [14] Paulo C Bartolomeu, Emanuel Vieira, Seyed M Hosseini, and Joaquim Ferreira. Self-sovereign identity: Use-cases, technologies, and challenges for industrial iot. In *2019 24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, pages 1173–1180. IEEE, 2019.
- [15] Bahar Houtan, Abdelhakim Senhaji Hafid, and Dimitrios Makrakis. A survey on blockchain-based self-sovereign patient identity in healthcare. *IEEE Access*, 8:90478–90494, 2020.
- [16] Reza Soltani, Uyen Trang Nguyen, and Aijun An. A survey of self-sovereign identity ecosystem. *Security and Communication Networks*, 2021(1):8873429, 2021.
- [17] Špela Čučko and Muhamed Turkanović. Decentralized and self-sovereign identity: Systematic mapping study. *IEEE Access*, 9:139009–139027, 2021.
- [18] Alexandre Siqueira, Arlindo Flavio Da Conceição, and Vladimir Rocha. Blockchains and self-sovereign identities applied to healthcare solutions: A systematic review. *arXiv preprint arXiv:2104.12298*, 2021.
- [19] Frederico Schardong and Ricardo Custódio. Self-sovereign identity: a systematic review, mapping and taxonomy. *Sensors*, 22(15):5641, 2022.
- [20] Md Rayhan Ahmed, AKM Muzahidul Islam, Swakkhar Shatabda, and Salekul Islam. Blockchain-based identity management system and self-sovereign identity ecosystem: A comprehensive survey. *IEEE Access*, 10:113436–113481, 2022.
- [21] Roberto Albeiro Pava Díaz, Rafael Vicente Páez Méndez, and Luis Fernando Niño Vásquez. A bibliometric study of scientific production on self-sovereign identity. *Ingeniería*, 28, 2023.
- [22] Geovane Fedrechski, Jan M Rabaey, Laisa CP Costa, Pablo C Calcina Ccori, William T Pereira, and Marcelo K Zuffo. Self-sovereign identity for iot environments: a perspective. In *2020 Global Internet of Things Summit (GloTS)*, pages 1–6. IEEE, 2020.
- [23] Ablyay Satybaldy, Md Sadek Ferdous, and Mariusz Nowostawski. A taxonomy of challenges for self-sovereign identity systems. *IEEE Access*, 2024.
- [24] Yue Liu, Qinghua Lu, Hye-Young Paik, and Xiwei Xu. Design patterns for blockchain-based self-sovereign identity. In *Proceedings of the European Conference on Pattern Languages of Programs 2020*, pages 1–14, 2020.
- [25] Špela Čučko, Vid Keršič, and Muhamed Turkanović. Towards a catalogue of self-sovereign identity design patterns. *Applied Sciences*, 13(9):5395, 2023.
- [26] Jayana Kaneriyi and Hiren Patel. A comparative survey on blockchain based self sovereign identity system. In *2020 3rd International Conference on Intelligent Sustainable Systems (ICISS)*, pages 1150–1155. IEEE, 2020.
- [27] Razieh Nokhbeh Zaeem, Kai Chih Chang, Teng-Chieh Huang, David Liau, Wenting Song, Aditya Tyagi, Manah Khalil, Michael Lamison, Siddharth Pandey, and K Suzanne Barber. Blockchain-based self-sovereign identity: Survey, requirements, use-cases, and comparative study. In *IEEE/WIC/ACM International Conference on Web Intelligence and Intelligent Agent Technology*, pages 128–135, 2021.
- [28] Špela Čučko, Šeila Bećirović, Aida Kamišalić, Saša Mrdović, and Muhamed Turkanović. Towards the classification of self-sovereign identity properties. *IEEE access*, 10:88306–88329, 2022.
- [29] Mohammed Shuaib, Noor Hafizah Hassan, Sahni Usman, Shadab Alam, Surbhi Bhatia, Arwa Mashat, Adarsh Kumar, and Manoj Kumar. Self-sovereign identity solution for blockchain-based land registry system: A comparison. *Mobile Information Systems*, 2022(1):8930472, 2022.
- [30] Yirui Bai, Hong Lei, Suozai Li, Haoyu Gao, Jun Li, and Leixiao Li. Decentralized and self-sovereign identity in the era of blockchain: a survey. In *2022 IEEE International Conference on Blockchain (Blockchain)*, pages 500–507. IEEE, 2022.
- [31] Andreas Grüner, Alexander Mühle, Niko Lockenvitz, and Christoph Meinel. Analyzing and comparing the security of self-sovereign identity management systems through threat modeling. *International Journal of Information Security*, 22(5):1231–1248, 2023.
- [32] Šeila Bećirović Ramić, Ehlimana Cogo, Irfan Prazina, Emir Cogo, Muhamed Turkanović, Razija Turčinodžić Mulahasanović, and Saša Mrdović. Selective disclosure in digital credentials: A review. *ICT Express*, 2024.

- [33] Manu Sporny, Dave Longley, Markus Sabadello, Drummond Reed, Orie Steele, and Christopher Allen. Decentralized identifiers (DIDs) v1.0 (W3C proposed recommendation 03 august 2021). Online <https://www.w3.org/TR/did-core/> [Accessed on 10 November 2025].
- [34] Christian Gribneau, Michael Prorock, Orie Steele, Terbu Oliver, Mike Xu, and Dmitri Zagidulin. did:web Method Specification. Online <https://w3c-ccg.github.io/did-method-web/> [Accessed on 10 November 2025].
- [35] Manu Sporny, Dmitri Zagidulin, Dave Longley, and Orie Steele. The did:key Method v0.7. Online <https://w3c-ccg.github.io/did-method-key/> [Accessed on 10 November 2025].
- [36] Ralph C Merkle. A digital signature based on a conventional encryption function. In *Conference on the theory and application of cryptographic techniques*, pages 369–378. Springer, 1987.
- [37] Gavin Wood et al. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper*, 151(2014):1–32, 2014.
- [38] Donald R Morrison. Patricia—practical algorithm to retrieve information coded in alphanumeric. *Journal of the ACM (JACM)*, 15(4):514–534, 1968.
- [39] Dan Boneh, Xavier Boyen, and Hovav Shacham. Short group signatures. In Matthew K. Franklin, editor, *Advances in Cryptology - CRYPTO 2004, 24th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 2004, Proceedings*, volume 3152 of *Lecture Notes in Computer Science*, pages 41–55. Springer, 2004.
- [40] T. Looker, V. Kalos, A. Whitehead, and M. Lodder. The bbs signature scheme. Online <https://identity.foundation/bbs-signature/draft-irtf-cfrg-bbs-signatures.html> [Accessed on 10 November 2025].
- [41] Paulo SLM Barreto, Ben Lynn, and Michael Scott. Constructing elliptic curves with prescribed embedding degrees. In *Security in Communication Networks: Third International Conference, SCN 2002 Amalfi, Italy, September 11–13, 2002 Revised Papers 3*, pages 257–267. Springer, 2003.
- [42] David Chadwick, Dave Longley, Manu Sporny, Olive Terbu, Dmitri Zagidulin, and Brent Zundel. Verifiable Credentials Implementation Guidelines 1.0. Online <https://www.w3.org/TR/vc-imp-guide/> [Accessed on 10 November 2025].
- [43] Rodelio Arenas and Proceso Fernandez. Credenceledger: a permissioned blockchain for verifiable academic credentials. In *2018 IEEE International Conference on Engineering, Technology and Innovation (ICE/ITMC)*, pages 1–6. IEEE, 2018.
- [44] Harry Halpin. Vision: A critique of immunity passports and w3c decentralized identifiers. In *Security Standardisation Research: 6th International Conference, SSR 2020, London, UK, November 30–December 1, 2020, Proceedings 6*, pages 148–168. Springer, 2020.
- [45] Anna Squicciarini, Elisa Bertino, Elena Ferrari, Federica Paci, and Bhavani Thuraisingham. Pp-trust-x: A system for privacy preserving trust negotiations. *ACM Transactions on Information and System Security (TISSEC)*, 10(3):12–es, 2007.
- [46] David W Kravitz. Exploration and impact of blockchain-enabled adaptive non-binary trust models. In *Proceedings of the 20th International Conference on Distributed Computing and Networking*, pages 124–133, 2019.
- [47] Anna C Squicciarini, Alberto Trombetta, Elisa Bertino, and Stefano Braghin. Identity-based long running negotiations. In *Proceedings of the 4th ACM workshop on Digital identity management*, pages 97–106, 2008.
- [48] Kenji Saito and Satoki Watanabe. Lightweight selective disclosure for verifiable documents on blockchain. *ICT Express*, 7(3):290–294, 2021.
- [49] Rahma Mukta, James Martens, Hye-young Paik, Qinghua Lu, and Salil S Kanhere. Blockchain-based verifiable credential sharing with selective disclosure. In *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 959–966. IEEE, 2020.
- [50] Aamna Tariq, Hina Binte Haq, and Syed Taha Ali. Cerberus: A blockchain-based accreditation and degree verification system. *IEEE Transactions on Computational Social Systems*, 10(4):1503–1514, 2022.
- [51] Ruijiao Tian, Lanju Kong, Baochen Zhang, Xiao Li, and Qingzhong Li. Authenticated selective disclosure of credentials in hybrid-storage blockchain. In *2022 IEEE 28th International Conference on Parallel and Distributed Systems (ICPADS)*, pages 330–337. IEEE, 2023.
- [52] José L Hernández-Ramos, Salvador Pérez, Christine Hennebert, Jorge Bernal Bernabé, Benoit Denis, Alexandre Macabies, and Antonio F Skarmeta. Protecting personal data in iot platform scenarios through encryption-based selective disclosure. *Computer Communications*, 130:20–37, 2018.
- [53] Alberto Sonnino, Mustafa Al-Bassam, Shehar Bano, Sarah Meiklejohn, and George Danezis. Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers. In *26th Annual Network and Distributed System Security Symposium, NDSS 2019, San Diego, California, USA, February 24-27, 2019*. The Internet Society, 2019.
- [54] Jan Camenisch and Anna Lysyanskaya. A signature scheme with efficient protocols. In Stelvio Cimato, Clemente Galdi, and Giuseppe Persiano, editors, *Security in Communication Networks, Third International Conference, SCN 2002, Amalfi, Italy, September 11-13, 2002. Revised Papers*, volume 2576 of *Lecture Notes in Computer Science*, pages 268–289. Springer, 2002.
- [55] Jan Camenisch, Manu Drijvers, and Anja Lehmann. Anonymous attestation using the strong diffie hellman assumption revisited. In *TRUST*, volume 9824 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2016.
- [56] Harika Narumanchi, Lakshmi Padmaja Maddali, and Nitesh Emmadi. Privacy enabled immunity credential system on blockchain. In *2022 14th International Conference on COMMunication Systems NETWORKS (COMSNETS)*, pages 1–6, 2022.
- [57] Matthias Babel and Johannes Sedlmeir. Bringing data minimization to digital wallets at scale with general-purpose zero-knowledge proofs. *arXiv preprint arXiv:2301.00823*, 2023.
- [58] Dan Yamamoto, Yuji Suga, and Kazue Sako. Formalising linked-data based verifiable credentials for selective disclosure. In *2022 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 52–65. IEEE, 2022.
- [59] Rui Shi, Yang Yang, Yingjiu Li, Huamin Feng, Guozhen Shi, Hwee Hwa Pang, and Robert H Deng. Double issuer-hiding attribute-based credentials from tag-based aggregatable mercurial signatures. *IEEE Transactions on Dependable and Secure Computing*, 2023.
- [60] Vasilis Kalos and George C Polyzos. Requirements and secure serialization for selective disclosure verifiable credentials. In *IFIP International Conference on ICT Systems Security and Privacy Protection*, pages 231–247. Springer, 2022.
- [61] Daniel Fett, Kristina Yasuda, and Brian Campbell. Selective Disclosure for JWTs (SD-JWT). Internet-Draft draft-ietf-oauth-selective-disclosure-jwt-05, Internet Engineering Task Force, June 2023. Work in Progress.
- [62] Hong Liu, Huansheng Ning, Yinliang Yue, Yueliang Wan, and Laurence T Yang. Selective disclosure and yoking-proof based privacy-preserving authentication scheme for cloud assisted wearable devices. *Future Generation Computer Systems*, 78:976–986, 2018.
- [63] Hugo Krawczyk, Mihir Bellare, and Ran Canetti. RFC2104: HMAC: keyed-hashing for message authentication, 1997.
- [64] National Institute of Standards, Technology, and Quynh Dang. The keyed-hash message authentication code (hmac). Federal Inf. Process. Stds. (NIST FIPS), July 2008. Revised version of FIPS 198 (2002).
- [65] Bjoern Czybik, Stefan Hausmann, Stefan Heiss, and Jürgen Jasperneite. Performance evaluation of mac algorithms for real-time ethernet communication systems. In *2013 11th IEEE International Conference on Industrial Informatics (INDIN)*, pages 676–681. IEEE, 2013.
- [66] Meltem Sönmez Turan and Luís TAN Brandão. Keyed-hash message authentication code (hmac): Specification of hmac and recommendations for message authentication. Online <https://doi.org/10.6028/NIST.SP.800-224.ipd> [Accessed on 10 November 2025], 2024.
- [67] John Kelsey, SJ Chang, and Ray Perlner. Sha-3 derived functions: cshake, kmac, tuplehash, and parallelhash. *NIST special publication*, 800:185, 2016.
- [68] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. *Available at SSRN 3440802*, 2008.
- [69] Ammar Mazri and Merouane Mehdi. Unraveling decentralized data storage: A comparative analysis of ipfs and bittorrent networks. In *2024 2nd International Conference on Electrical Engineering and Automatic Control (ICEEAC)*, pages 1–6. IEEE, 2024.
- [70] Ohad Rodeh and Avi Teperman. zfs-a scalable distributed file system using object disks. In *20th IEEE/11th NASA Goddard Conference on Mass Storage Systems and Technologies, 2003.(MSST 2003). Proceedings.*, pages 207–218. IEEE, 2003.
- [71] Andrea Flamini, Giada Sciarretta, Mario Scuro, Amir Sharif, Alessandro Tomasi, Silvio Ranise, et al. On cryptographic mechanisms for the selective disclosure of verifiable credentials. *JOURNAL OF INFORMATION SECURITY AND APPLICATIONS*, 83, 2024.
- [72] Tobias Looker, Vasilis Kalos, Andrew Whitehead, and Mike Lodder. The BBS Signature Scheme. Internet-Draft draft-irtf-cfrg-bbs-signatures-09, Internet Engineering Task Force, July 2025. Work in Progress.
- [73] Michael Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC 7519, May 2015.

- [74] Credentials Community Group (W3C). Linked data cryptographic suite registry (draft community group report 29 december 2020). Online <https://w3c-ccg.github.io/ld-cryptosuite-registry/> [Accessed on 10 November 2025].
- [75] uPort. Ethr-DID Library. Online <https://github.com/uport-project/ethr-did> [Accessed on 10 November 2025].
- [76] Decentralized Identity Foundation. ethr DID Resolver. Online <https://github.com/decentralized-identity/ethr-did-resolver/> [Accessed on 10 November 2025].
- [77] Stefano Tessaro and Chenzhi Zhu. Revisiting BBS signatures. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V*, volume 14008 of *Lecture Notes in Computer Science*, pages 691–721. Springer, 2023.
- [78] Man Ho Au, Willy Susilo, and Yi Mu. Constant-size dynamic k -taa. In Roberto De Prisco and Moti Yung, editors, *Security and Cryptography for Networks, 5th International Conference, SCN 2006, Maiori, Italy, September 6-8, 2006, Proceedings*, volume 4116 of *Lecture Notes in Computer Science*, pages 111–125. Springer, 2006.
- [79] Decentralized Identity Foundation. did-jwt-vc. Online, <https://github.com/decentralized-identity/did-jwt-vc> [Accessed on 10 November 2025].
- [80] National Institute of Standards, Technology (NIST), and Morris J. Dworkin. Sha-3 standard: Permutation-based hash and extendable-output functions, 2015-08-04 00:08:00 2015.
- [81] Committee on National Security Systems. Cnss policy no. 15, fact sheet no. 1: National policy on the use of the advanced encryption standard (aes). Fact Sheet / Policy Fact Sheet No. 1, CNSS Policy No. 15, National Security Agency, June 2003. Public domain (U.S. government work).
- [82] Daniel J Bernstein and Tanja Lange. ebats: Ecrypt benchmarking of asymmetric systems. <http://bench.cr.yp.to> [Accessed on 10 November 2025], 2017.
- [83] J Alakuijala et al. Brotli compressed data format draft-alakuijala-brotli-08. *Network Working Group, dated Dec, 2015*.

Andrea De Salve is a researcher at the Institute of Applied Sciences and Intelligent Systems (ISASI) under the National Research Council (CNR) since 2020. From 2017 to 2019, he was a Research Assistant with the University of Palermo and the Institute of Informatics and Telematics (IIT), where he contributed to projects focused on data privacy and security. With a PhD in Computer Science from the University of Pisa, Andrea has a strong background in privacy and security of data in decentralized online social networks. He obtained a PhD in Computer Science from the University of Pisa. His academic contributions include numerous publications in prestigious journals and conferences, and he has been actively involved in various national and international research projects, contributed to the development of innovative solutions for smart contracts, blockchain technology, and IoT security. He has served on technical program committees for several international conferences and has been a reviewer for various scientific journals.

Andrea Lisi Andrea Lisi received his Ph.D. degree in computer science in 2023 at the University of Pisa, Italy, department of computer science, with his thesis titled "Designing layer-2 blockchain applications: methodologies and solutions". His research interests include decentralized technologies, such as blockchain, and Self-Sovereign Identity.

Maurizio Cascino graduated in Computer Science from the University of Pisa and currently works as an IT consultant in the banking sector. Maurizio is deeply drawn to the intricate world of cybersecurity, a passion that has driven him to pursue advanced academic studies by enrolling in a specialized master's program in cybersecurity. The choice demonstrates his interest to cybersecurity.

Paolo Mori is a senior researcher of the Trust, Security, and Privacy research unit at the Institute of Informatics and Telematic (IIT) of the Italian National Research Council (CNR). He has a PhD in Computer Science from the University of Pisa. His main research interests involve trust, security and privacy in distributed systems, focusing on the design and implementation of mechanisms for access/usage control and for trust and reputation management in distributed environments, such as the Cloud and the Internet of Things. He is also interested in Blockchain technology and its applications, focusing on security aspects. He usually serves in the committees of international conferences and workshops. He is general chair of the "11th International Conference of Information System Security and Privacy" (ICISSP 2025) and of the "7th IEEE International Conference on Blockchain and Cryptocurrency" (ICBC 2025). He is (co-)author of 150+ papers published on international journals and conference/workshop proceedings, and of several chapters in books on research topics. He is usually actively involved in research projects on information and communication security funded by the European Commission. He is also actively involved in SERICS, a project under the MUR National Recovery and Resilience Plan funded by the European Union - NextGenerationEU.

Laura Ricci is a full professor at the University of Pisa, where she also received her Ph.D. in Computer Science. Her research interests encompass cryptocurrencies, blockchains, Web3 and the metaverse, as well as social network analysis. She has authored over 150 publications in international journals and conference/workshop proceedings. She has served in several leadership roles, including as the local coordinator of the H2020 European Helios project and the principal investigator of the Italian National Project AWESOME: Analysis framework for Web3 Social Media. Additionally, she is the co-chair of the BRAIN workshop on Blockchain Theory and Applications and has been a TPC co-chair and chair for various international conferences on blockchain technology. Her expertise has been recognized at the national level, having been a member of the Italian national commission responsible for defining a national blockchain strategy, as well as a member of the Bank of Italy committee for evaluating proposals related to Distributed Ledger Technology.

...